



## Proyecto Final

Prof Carmen Lucia Bustillo Hernández  
carmenbustillohernandez@gmail.com



Facultad de Ingeniería  
**Asignatura: Sistemas Distribuidos**  
Universidad Nacional Autónoma de México  
Fecha de Entrega: 05 de junio de 2026

**Nombre:** Enrique Yoab Cortez Rios

**Matrícula:** 317302992

**Nombre:** Mario Mendoza González

**Matrícula:** 316284828

### Objetivo General

El alumno comprenderá y aplicará los mecanismos de intercomunicación de procesos y concurrencia a bajo nivel mediante el desarrollo de una aplicación distribuida peer-to-peer con interfaz gráfica.

### Objetivos Específicos

1. Diseñar e implementar una aplicación tipo chat (messenger) multiplataforma utilizando programación visual.

### Material

1. Equipo de cómputo con sistema operativo Windows. Mac o Linux y acceso a Internet.
2. Editor de textos (MS Word, Libre Office).
3. Compilador o IDE de lenguajes de programación.

### Bibliografía

1. Tanenbaum, A. S. (2004). Sistemas operativos modernos. En Prentice-Hall, Inc eBooks .  
<http://dl.acm.org/citation.cfm?id=120575>

## 1. Instrucciones Generales

1. Realizar el proyecto con las indicaciones que se solicitan.
2. Para el proyecto, desarrolle su diagrama de flujo y pseudocódigo respectivamente.

## 2. Criterios de Evaluación

1. Se evaluará de acuerdo con los Criterios de Evaluación que se encuentra en el Google Classroom de la asignatura.

## 3. Observaciones Previas

- **Número de Procesos:** El sistema requiere la bifurcación de dos procesos independientes mediante la directiva `fork()`. El proceso hijo actúa como el *backend*, encargándose de gestionar el tráfico de red en tiempo real a través de una arquitectura *peer-to-peer* (P2P) basada en sockets TCP/IP. El proceso padre opera como el *frontend*, administrando el bucle principal de la interfaz gráfica de usuario (GUI) y capturando las interacciones del usuario mediante la biblioteca GTK.
- **Cantidad de Tuberías:** Se implementan dos tuberías unidireccionales (*pipes*) interconectadas de forma cruzada para establecer un canal de comunicación dúplex completo (*Full-Duplex*) entre ambos procesos. Este mecanismo de comunicación entre procesos (*IPC*) es fundamental para el intercambio asíncrono de estructuras de datos tipo **Mensaje** entre la lógica de red y la interfaz visual.
- **Hilos de Ejecución (*threads*):** La concurrencia interna se gestiona mediante hilos especializados para evitar bloqueos en el flujo de datos:
  - **Proceso de Red (*Backend*):** Consta de 4 hilos distribuidos de la siguiente manera: un hilo servidor bloqueante para la escucha y aceptación de conexiones entrantes, un hilo escritor orientado a la transmisión de datos hacia el nodo remoto, un hilo lector síncrono asignado a la recepción del pipe proveniente de la GUI y un hilo escritor dedicado a canalizar los mensajes de red entrantes hacia la interfaz gráfica.
  - **Proceso Gráfico (*Frontend*):** Implementa únicamente 2 hilos asíncronos dedicados de forma exclusiva a las tareas de lectura y escritura sobre las tuberías compartidas, garantizando que la renderización de la interfaz no sufra retardos.

## 4. Desarrollo del Proyecto

### 4.1 Archivo de Cabecera (`header.h`)

Este archivo contiene las declaraciones de las estructuras de datos y la definición de las funciones utilizadas para la implementación del proyecto.

- **Estructura de Datos:** Se definen 2 estructuras principales; `Mensaje` y `TipoMensaje`. La primera encapsula la información relevante de cada mensaje, el id del proceso que lo envía, el contenido del mensaje, la IP y el puerto del nodo remoto, así como el tipo de mensaje que se está transmitiendo. La segunda estructura, es un enumerado que categoriza los mensajes en tres tipos; `ENVIO`, `RECIBIDO` y `SALIDA`, lo que permite una gestión eficiente de los flujos de datos y la lógica de control entre ambos procesos.
- **Funciones del Back-End:** se usan 8 funciones para el proceso de red, cada una con un rol específico en la gestión de la comunicación y la concurrencia. Estas funciones incluyen los hilos lectores y escritores tanto para la comunicación P2P como para las tuberías. Y dos funciones auxiliares para capturar el tiempo en milisegundos y otro para convertirlo a formato de texto.
- **Funciones del Front-End / Interfaz:** Se definen 3 funciones destinadas a la gestión de la interfaz gráfica de usuario y su interacción asíncrona con las tuberías de comunicación interprocesos. Estas comprenden un hilo lector especializado en recuperar los datos entrantes del *backend* para delegar su renderizado en burbujas sin bloquear la fluidez de la ventana visual, un hilo escritor responsable de monitorizar y canalizar los mensajes de salida capturados desde la entrada de texto, y la rutina maestra de inicialización encargada de estructurar el entorno gráfico con GTK, instanciar la concurrencia del *frontend* y desplegar los componentes en pantalla.

#### 4.1.1 Código

Listing 1: Archivo de declaraciones compartidas entre el Backend y el Frontend

```
1 #ifndef HEADER_H
2 #define HEADER_H
3 #include <gtk/gtk.h>
4
5 #define TAM_MAX 1024
6
7 typedef enum {
8     ENVIO,
9     RECEPCION,
10    SALIDA
11 } TipoMensaje;
12
13 typedef struct {
```

```
14     int id_emisor;
15     char texto[TAM_MAX];
16     char ip_destino[16];
17     int puerto_destino;
18     TipoMensaje tipo;
19     long long tiempo;
20 } Mensaje;
21
22 typedef struct {
23     int A[2]; // Interfaz <-- Backend (Backend escribe en A[1], Interfaz lee en A[0])
24     int B[2]; // Interfaz --> Backend (Interfaz escribe en B[1], Backend lee en B[0])
25 } Tuberia;
26
27 // Prototipos del Backend (conexion.c)
28 void *hilo_lector_p2p(void *arg);
29 void *hilo_escritor_p2p(void *arg);
30 void *hilo_lector_pipe(void *arg);
31 void *hilo_escritor_pipe(void *arg);
32 void conexion(Tuberia *local);
33 void enviar_msj(char *ip_destino, int puerto_destino, Mensaje msg);
34 long long get_time_ms();
35 void formatear_tiempo_str(long long tiempo_ms, char *buffer_salida);
36
37 // Prototipos del Frontend / Interfaz (interfaz.c)
38 void *hilo_lector_gui(void *arg);
39 void *hilo_escritor_gui(void *arg);
40 void interfaz_grafica(int argc, char *argv[], Tuberia *padre);
41
42 #endif
```

#### 4.1.2 Pseudocódigo (header.h)

```
1 // Guardas de seguridad para evitar la doble inclusión del archivo en el compilador
2 Si NO está definido HEADER_H Entonces
3     Definir HEADER_H
4
5     // Configuración y Constantes Globales
6     Constante TAM_MAX <- 1024
7
8     // ===== DEFINICIÓN DE TIPOS DE DATOS =====
9
10    // Enumeración para clasificar la naturaleza del mensaje IPC/Red
11    Enumeración TipoMensaje:
```

```
12      ENVIO          // Mensaje saliente generado por el usuario local
13      RECEPCION     // Mensaje entrante proveniente del nodo remoto
14      SALIDA        // Evento de control para apagar los hilos y procesos
15
16  // Estructura principal que empaqueta un mensaje de chat
17  Estructura Mensaje:
18      id_emisor AS Entero          // PID del proceso o identificador de nodo
19      texto AS Arreglo[TAM_MAX] de Caracter // Cuerpo del mensaje de texto plano
20      ip_destino AS Arreglo[16] de Caracter // Dirección IP remota (IPv4)
21      puerto_destino AS Entero      // Puerto de red remoto
22      tipo AS TipoMensaje          // Clasificación (ENVIO, RECEPCION, SALIDA)
23      tiempo AS Entero_Grande      // Marca de tiempo en milisegundos (Unix
timestamp)
24
25  // Estructura que agrupa los canales FIFO (Pipes) de comunicación bidireccional
26  Estructura Tuberia:
27      A AS Arreglo[2] de Entero // Canal de Entrada a la GUI: Backend (Escribe A[1])
-> Interfaz (Lee A[0])
28      B AS Arreglo[2] de Entero // Canal de Salida de la GUI: Interfaz (Escribe B[1])
-> Backend (Lee B[0])
29
30
31  // ===== PROTOTIPOS DEL BACKEND (conexion.c) =====
32
33  Funcion hilo_lector_p2p(arg AS Puntero) Devuelve Puntero
34  Funcion hilo_escritor_p2p(arg AS Puntero) Devuelve Puntero
35  Funcion hilo_lector_pipe(arg AS Puntero) Devuelve Puntero
36  Funcion hilo_escritor_pipe(arg AS Puntero) Devuelve Puntero
37
38  Procedimiento conexion(local AS Referencia a Tuberia)
39  Procedimiento enviar_msj(ip_destino AS Cadena, puerto_destino AS Entero, msg AS
Mensaje)
40
41  Funcion get_time_ms() Devuelve Entero_Grande
42  Procedimiento formatear_tiempo_str(tiempo_ms AS Entero_Grande, buffer_salida AS
Referencia a Cadena)
43
44
45  // ===== PROTOTIPOS DEL FRONTEND (interfaz.c) =====
46
47  Funcion hilo_lector_gui(arg AS Puntero) Devuelve Puntero
48  Funcion hilo_escritor_gui(arg AS Puntero) Devuelve Puntero
49
```

```
50  Procedimiento interfaz_grafica(argc AS Entero, argv[] AS Cadena, padre AS Referencia  
    a Tuberia)  
51  
52 Fin Si
```

## 4.2 Archivo de Conexión (conexion.c)

Este archivo es el encargado de contener toda la lógica de la red P2P, actuando como un proceso independiente (*backend*) para aislar la comunicación y evitar problemas de sincronización o bloqueos en la interfaz gráfica (GUI). Implementa una arquitectura multihilo para gestionar simultáneamente el tráfico de red mediante sockets TCP/IP y la comunicación interprocesos (IPC) mediante tuberías. A continuación, se describe a detalle el propósito y funcionamiento de cada subrutina contenida en este módulo:

- **conexion()**: Es la función orquestadora del *backend*. Se encarga de instanciar y levantar los 4 hilos principales de ejecución mediante la directiva `pthread_create`. Además, utiliza `pthread_join` para mantener el proceso activo y garantizar que todos los hilos se unan y se destruyan de forma segura al momento de cerrar la aplicación.
- **hilo\_lector\_p2p()**: Actúa como el servidor pasivo de la arquitectura P2P. Configura un socket TCP, lo enlaza (`bind`) al puerto local y se queda a la escucha (`listen`). Cuando recibe una conexión entrante a través de `accept()`, extrae el flujo binario hacia la estructura `Mensaje` y activa la bandera volátil `regresa` para notificar que hay datos listos.
- **hilo\_escritor\_p2p()**: Funciona como el cliente activo de la red. Ejecuta un ciclo continuo evaluando la bandera volátil `escribe`. Cuando esta se enciende (indicando que la interfaz ha depositado un mensaje), extrae la información del buffer compartido y despacha el paquete hacia la IP y puerto del nodo remoto.
- **hilo\_lector\_pipe()**: Es el puente de entrada desde el *frontend*. Permanece bloqueado leyendo el extremo de lectura del Pipe B (`local->B[0]`). Al recibir un paquete proveniente de la GUI, evalúa si es un texto ordinario o un evento de cierre (`id_emisor == -1`), en cuyo caso inyecta un mensaje local para iniciar la secuencia de apagado global.
- **hilo\_escritor\_pipe()**: Es el puente de salida hacia la interfaz. Monitorea la bandera que levanta el servidor P2P y, al activarse, inyecta el mensaje recibido por la red en el extremo de escritura del Pipe A (`local->A[1]`) para que la vista gráfica pueda renderizarlo dinámicamente.
- **enviar\_msj()**: Función que encapsula la rutina de transmisión TCP. Crea un descriptor temporal, adapta la IP y el puerto a formato de red con `inet_pton` y `htons`, establece una conexión activa de tres vías (`connect()`) con el destinatario, transmite la estructura `Mensaje` en crudo y libera el socket.

- **Funciones de Tiempo (`get_time_ms` y `formatear_tiempo_str`):** Subrutinas de apoyo encargadas de capturar la hora del sistema operativo con precisión de milisegundos y darle formato de cadena (`HH:MM:SS:ms`) para su posterior visualización en las burbujas del chat.

Listing 2: Archivo del Back-End (conexion P2P)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6 #include <netinet/in.h>
7 #include <sys/wait.h>
8 #include <pthread.h>
9 #include <sys/time.h>
10 #include "header.h"
11
12 // Variable que controla el ciclo de vida de los hilos de la conexion P2P
13 volatile int encendido = 1;
14 // Variable que controla el ciclo de vida de los hilos de los pipes
15 volatile int activo = 1;
16 // Variable que controla la accion de escribir por la Red (socket)
17 volatile int escribe = 0;
18 // Variable que controla la accion de escribir para la UI (pipes)
19 volatile int regresa = 0;
20
21 int PUERTO = 4000; // El puerto que usamos nosotros
22 char DIRECTION[] = "127.0.0.1"; // No se cambia la ip de nuestra compu
23
24 // Mensaje que se recibe por la Interfaz.
25 Mensaje interfaz;
26 // Mensaje que se recibe por el Socket.
27 Mensaje externo;
28
29 // Funcion que se encarga de crear los 4 hilos de la conexion P2P
30 void conexion(Tuberia *local)
31 {
32     // Se crean 4 variables tipo hilo
33     pthread_t servidor, escritor, pipe_lector, pipe_escritor;
34
35     printf("[HIJO] Inicializando los 4 hilos (Sockets y Pipes)...\n");
36     // Creamos el hilo servidor (P2P)
37     pthread_create(&servidor, NULL, hilo_lector_p2p, NULL);
38     // Creamos el hilo cliente (P2P)
```

```
39 pthread_create(&escritor, NULL, hilo_escritor_p2p, NULL);
40 // Creamos el hilo lector (pipe B)
41 pthread_create(&pipe_lector, NULL, hilo_lector_pipe, (void *)local);
42 // Creamos el hilo escritor (pipe A)
43 pthread_create(&pipe_escritor, NULL, hilo_escritor_pipe, (void *)local);
44
45 // Esperamos a los 4 hilos creados para finalizar correctamente en orden
46 pthread_join(servidor, NULL);
47 pthread_join(escritor, NULL);
48 pthread_join(pipe_lector, NULL);
49 pthread_join(pipe_escritor, NULL);
50
51 printf("[HIJO] Todos los hilos destruidos de forma segura. Saliendo.\n");
52 }
53
54 // Hilo que se encarga de escuchar mensajes por el socket y nuestro PUERTO
55 void *hilo_lector_p2p(void *arg)
56 {
57     // Descriptores de archivo: server_fd para la escucha y new_socket para la conexion
    aceptada
58     int server_fd, new_socket;
59
60     // Estructura de red que almacenara los datos de direccionamiento IP y Puerto del
    servidor
61     struct sockaddr_in address;
62
63     // Variable de configuracion para habilitar opciones en el nivel de socket
64     int opt = 1;
65
66     // Estructura local donde se desempaquetara el mensaje binario recibido por la red
67     Mensaje msg_recibido;
68
69     // Intentamos crear un socket de flujo (TCP) utilizando el protocolo de internet
    IPv4
70     if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0)
71     {
72         // Si la creacion del socket falla, terminamos el hilo de forma segura
        retornando NULL
73         pthread_exit(NULL);
74     }
75
76     // Permite reutilizar el puerto inmediatamente si la aplicacion se reinicia,
    evitando el error "Address already in use"
```



```
77     setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
78
79     // Indicamos que utilizaremos la familia de direcciones IPv4
80     address.sin_family = AF_INET;
81
82     // Vinculamos el servidor a cualquier interfaz de red disponible en el equipo
83     // (0.0.0.0)
84     address.sin_addr.s_addr = INADDR_ANY;
85
86     // Asignamos el puerto de escucha (4000) convirtiendolo al formato de red (Big
87     // Endian) con htons
88     address.sin_port = htons(PUERTO);
89
90     // Asociamos la configuracion de la IP y el PUERTO al descriptor de archivo del
91     // socket
92     if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0)
93     {
94         // Si el puerto ya esta ocupado o falla el enlace, cerramos el descriptor y
95         // salimos del hilo
96         close(server_fd);
97         pthread_exit(NULL);
98     }
99
100    // Configuramos el socket en modo pasivo, permitiendo una cola de hasta 5 conexiones
101    // en espera
102    if (listen(server_fd, 5) < 0)
103    {
104        // Si el sistema operativo no puede asignar la cola, liberamos el recurso y
105        // salimos
106        close(server_fd);
107        pthread_exit(NULL);
108    }
109
110    // Ciclo de vida del hilo servidor (P2P); se ejecutara activamente mientras la
111    // bandera global sea verdadera
112    while (encendido)
113    {
114        // Obtenemos el tamaño de la estructura de direccionamiento para la llamada
115        // accept
116        int tam_addr = sizeof(address);
117
118        // Bloquea el hilo aqui hasta que un nodo remoto intente conectarse, generando
119        // un nuevo socket especifico
```

```
111     new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&
112     tam_addr);
113
114     // Si la conexion se establecio correctamente y el descriptor es valido
115     if (new_socket >= 0)
116     {
117         // Limpiamos la memoria del buffer local de recepcion para evitar basura de
118         // datos anteriores
119         memset(&msg_recibido, 0, sizeof(Mensaje));
120
121         // Leemos el flujo binario proveniente del socket remoto y lo mapeamos
122         // directo en la estructura Mensaje
123         read(new_socket, &msg_recibido, sizeof(Mensaje));
124
125         // Si el id_emisor es -1, significa que el nodo remoto o la interfaz local
126         // envio un evento de cierre
127         if (msg_recibido.id_emisor == -1)
128         {
129             // Desplegamos en consola el evento global de finalizacion del programa
130             printf("[GLOBAL] Evento de cierre... Cerrando hilos...\n");
131
132             // Apagamos las banderas de control del backend para romper el ciclo de
133             // los demas hilos
134             encendido = 0;
135             activo = 0;
136
137             // Cerramos de forma segura las conexiones
138             close(new_socket);
139
140             // Rompemos el ciclo infinito de escucha activa
141             break;
142         }
143         // Si el mensaje es ordinario (contiene texto de un chat activo)
144         else
145         {
146             // Informamos que se recibio con exito un paquete de datos en el puerto
147             // local configurado
148             printf("Hilo recibio mensaje en %d...\n", PUERTO);
149
150             // Forzamos el tipo de mensaje a RECEPCION para que la GUI sepa que debe
151             // pintarlo a la izquierda
152             msg_recibido.tipo = RECEPCION;
```

```
147         // Copiamos el contenido de forma segura a la variable global compartida
'externo'
148         memcpy(&externo, &msg_recibido, sizeof(Mensaje));
149
150         // Activamos la bandera volatil para indicarle al hilo escritor del pipe
que hay datos listos
151         regresa = 1;
152     }
153     // Cerramos el socket de la sesion actual para liberar el descriptor una vez
procesado el mensaje
154     close(new_socket);
155 }
156 }
157 // Al salir del ciclo principal, cerramos definitivamente el socket maestro del
servidor
158 close(server_fd);
159
160 // Terminamos la ejecucion del hilo de forma limpia ante el entorno de pthreads
161 pthread_exit(NULL);
162 }
163
164 // Hilo que se encarga de escribir a un PUERTO e IP enviado por la interfaz
165 void *hilo_escritor_p2p(void *arg)
166 {
167     // Ciclo de vida del hilo cliente (P2P); se ejecutara activamente mientras la
bandera global sea verdadera
168     while (encendido)
169     {
170         // Si la bandera cambia es que recibio un mensaje de la interfaz y debe enviarlo
al p2p amigo
171         if (escribe == 1)
172         {
173             // Se desactiva la bandera ya que solo debe realizarse una vez.
174             escribe = 0;
175             // Envia el mensaje que ya se cargo en la estructura de Mensaje interfaz;
176             enviar_msj(interfaz.ip_destino, interfaz.puerto_destino, interfaz);
177             // Imprime en pantalla que envio un mensaje (Depuracion)
178             printf("Hilo envio mensaje a %d...\n", interfaz.puerto_destino);
179         }
180         // Al ser un evento volatil se debe dormir considerablemente para evitar el
consumo excesivo del CPU
181         usleep(1000);
182     }
```

```
183 // Sale del hilo para terminar la ejecucion.
184 pthread_exit(NULL);
185 }
186
187 // Hilo que se encarga de interceptar el mensaje que envia la interfaz grafica por el
    pipe B
188 void *hilo_lector_pipe(void *arg)
189 {
190     // El hilo tiene como entrada la estructura de Tuberia que tiene los pipes a
    utilizar
191     Tuberia *local = (Tuberia *)arg;
192     // Variable para almacenar el mensaje y despues copiarlo a la variable volatil
    interfaz
193     Mensaje msg_pipe;
194
195     // Ciclo de vida del hilo lector (pipe B); Se ejecuta activamente mientras la
    bandera global sea verdadera.
196     while (activo)
197     {
198         // Al ser un evento bloqueante se queda esperando hasta recibir algo de la
    interfaz
199         int bytes = read(local->B[0], &msg_pipe, sizeof(Mensaje));
200         // Si el mensaje si contiene algo y sigue activo la bandera global evalua el
    mensaje recibido.
201         if (bytes > 0 && activo)
202         {
203             // Si el id del mensaje es -1 significa que es un eveno de cierre y debe
    avisar a la conexion P2P
204             if (msg_pipe.id_emisor == -1)
205             {
206                 // Inyecta el mensaje a si mismo para que lo procese
207                 enviar_msj(DIRECTION, PUERTO, msg_pipe);
208                 // Rompemos el ciclo ya que se inyecto el mensaje de cierre y la
    conexion cierra los demas hilos
209                 break;
210             }
211             else // Si el id no es -1 es del usuario y debemos activar al cliente de la
    conexion P2P
212             {
213                 // Muestra un mensaje en la interfaz de que se recibio un mensaje del
    usuario
214                 printf("Recibi Mensaje de la GUI...\n");
215                 // Copiamos el mensaje recibido a la variable global interfaz
```

```
216         memcpy(&interfaz, &msg_pipe, sizeof(Mensaje));
217         // Activamos la bandera de escritura para el P2P (cliente)
218         escribe = 1;
219     }
220 }
221 }
222 pthread_exit(NULL);
223 }
224
225 // Hilo que se encarga de escribir hacia la interfaz grafica por el pipe A
226 void *hilo_escritor_pipe(void *arg)
227 {
228     // El hilo tiene como entrada la estructura de Tuberia que tiene los pipes a
229     // utilizar
230     Tuberia *local = (Tuberia *)arg;
231     // Ciclo de vida del hilo escritor (pipe A); Se ejecuta activamente mientras la
232     // bandera global sea verdadera.
233     while (activo)
234     {
235         // Si la bandera cambio significa que la conexion P2P (servidor) recibio un
236         // mensaje por lo que debemos enviarlo a la interfaz por el pipe
237         if (regresa == 1)
238         {
239             // Desactivamos la bandera ya que solo debe realizarse una vez
240             regresa = 0;
241             // Escribe a la tuberia unicamente (pipe A)
242             write(local->A[1], &externo, sizeof(Mensaje));
243         }
244         // Al ser un evento volatil se debe dormir considerablemente para evitar el
245         // consumo excesivo del CPU
246         usleep(1000);
247     }
248     pthread_exit(NULL);
249 }
250
251 // Funcion para enviar el Mensaje a una IP y PUERTO especifico (Cliente TCP).
252 void enviar_msj(char *ip_destino, int puerto_destino, Mensaje msg)
253 {
254     // Descriptor de archivo para el socket local que iniciará la conexión.
255     int sock = 0;
256
257     // Estructura de red que almacenará la dirección IP y el Puerto del nodo remoto (
258     // destino).
```

```
254 struct sockaddr_in serv_addr;
255
256 // Intentamos crear un socket de flujo activo utilizando el protocolo TCP/IP bajo
257 // IPv4.
258 if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0)
259     // Si el sistema operativo no puede asignar el socket, salimos de la función
260     // inmediatamente.
261     return;
262
263 // Convertimos el puerto destino de formato de host a formato de red (Big Endian)
264 // usando htons.
265 serv_addr.sin_port = htons(puerto_destino);
266
267 // Especificamos que la dirección de destino pertenece a la familia IPv4.
268 serv_addr.sin_family = AF_INET;
269
270 // Convertimos la IP de texto (ej. "127.0.0.1") a su representación binaria de red.
271 if (inet_pton(AF_INET, ip_destino, &serv_addr.sin_addr) <= 0)
272 {
273     // Si la IP de destino es inválida o el formato es incorrecto, cerramos el
274     // socket y salimos.
275     close(sock);
276     return;
277 }
278
279 // Intentamos establecer una conexión activa de tres vías (Three-way handshake) con
280 // el nodo remoto.
281 if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) >= 0)
282 {
283     // Si la conexión es exitosa, enviamos la estructura completa 'Mensaje' de
284     // manera directa por el flujo del socket.
285     send(sock, &msg, sizeof(Mensaje), 0);
286 }
287
288 // Cerramos el socket una vez terminada la transferencia (o si falló la conexión)
289 // para liberar el descriptor.
290 close(sock);
291 }
292
293 // Función para obtener el tiempo en milisegundos
294 long long get_time_ms()
295 {
296     struct timeval tv;
```

```
290     gettimeofday(&tv, NULL);
291     return (long long)(tv.tv_sec) * 1000 + (tv.tv_usec) / 1000;
292 }
293
294 // Función para transformar ms a HH:MM:SS:ms
295 void formatear_tiempo_str(long long tiempo_ms, char *buffer_salida)
296 {
297     time_t segundos = tiempo_ms / 1000;
298     int milisegundos = tiempo_ms % 1000;
299     struct tm *tm_info = localtime(&segundos);
300
301     sprintf(buffer_salida, "%02d:%02d:%02d:%03d", tm_info->tm_hour, tm_info->tm_min,
302             tm_info->tm_sec, milisegundos);
303 }
```

#### 4.2.1 Pseudocodigo (conexion.c)

```
1 // Banderas de control de ciclos de vida
2 Volatil Entero encendido <- 1
3 Volatil Entero activo <- 1
4
5 // Banderas de control de eventos de escritura
6 Volatil Entero escribe <- 0
7 Volatil Entero regresa <- 0
8
9 // Configuración de red por defecto
10 Entero PUERTO <- 4000
11 Cadena DIRECCION <- "127.0.0.1"
12
13 // Buffers globales donde se comparten datos entre hilos
14 Mensaje interfaz // Datos desde la GUI
15 Mensaje externo // Datos desde la Red (Sockets)
16
17 Procedimiento conexion(local AS Referencia a Tuberia):
18     Imprimir "[HIJO] Inicializando los 4 hilos (Sockets y Pipes)..."
19
20     // Crear los hilos y asignarles sus respectivas funciones de manera concurrente
21     CREAR_HILO(servidor, hilo_lector_p2p, Nulo)
22     CREAR_HILO(escriptor, hilo_escritor_p2p, Nulo)
23     CREAR_HILO(pipe_lector, hilo_lector_pipe, local)
24     CREAR_HILO(pipe_escritor, hilo_escritor_pipe, local)
25
26     // Esperar de forma bloqueante a cada uno hilo
```

```
27  ESPERAR_HILO(servidor)
28  ESPERAR_HILO(escritor)
29  ESPERAR_HILO(pipe_lector)
30  ESPERAR_HILO(pipe_escritor)
31
32  Imprimir "[HIJO] Todos los hilos destruidos de forma segura. Saliendo."
33 Fin Procedimiento
34
35 Funcion hilo_lector_p2p(arg AS Puntero):
36     server_fd <- CREAR_SOCKET(AF_INET, SOCK_STREAM) // Protocolo TCP
37     Si server_fd < 0 Entonces TERMINAR_HILO()
38
39     CONFIGURAR_SOCKET(server_fd, SO_REUSEADDR) // Reutilizar puerto rápido
40
41     // Configurar los datos del Servidor
42     Dirección_Red.familia <- AF_INET
43     Dirección_Red.ip <- CUALQUIERA_DISPONIBLE
44     Dirección_Red.puerto <- CONVERTIR_A_FORMATO_RED(PUERTO)
45
46     Si VINCULAR_PUERTO(server_fd, Dirección_Red) < 0 Entonces
47         CERRRAR_SOCKET(server_fd)
48         TERMINAR_HILO()
49     Fin Si
50
51     ESCUCHAR_PUERTO(server_fd, Limite_Cola <- 5)
52
53     msg_recibido AS Mensaje
54
55     Mientras encendido == 1 Hacer:
56         // Esperamos una conexion externa
57         new_socket <- ACEPTAR_CONEXION(server_fd, Dirección_Red)
58
59         Si new_socket >= 0 Entonces
60             LIMPIAR_MEMORIA(msg_recibido)
61             LEER_DE_SOCKET(new_socket, msg_recibido)
62
63         // Caso_1: Evento de apagado del programa
64         Si msg_recibido.id_emisor == -1 Entonces
65             Imprimir "[GLOBAL] Evento de cierre... Cerrando hilos..."
66             encendido <- 0
67             activo <- 0
68             CERRRAR_SOCKET(new_socket)
69             Romper Bucle
```



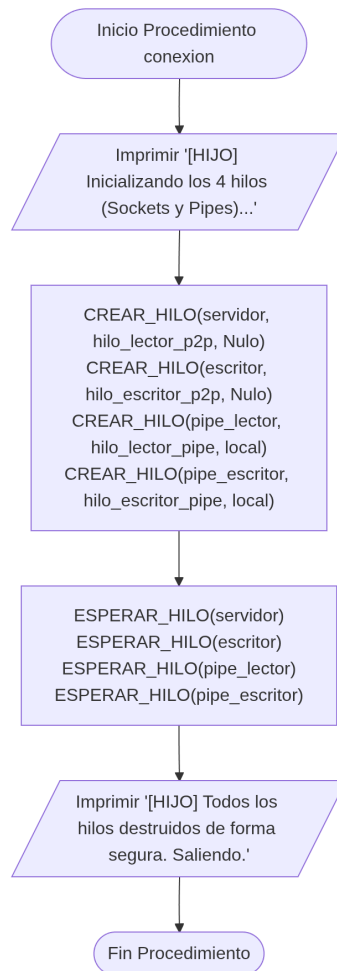
```
70
71     // Caso_2: Mensaje de chat normal recibido
72     Sino
73         Imprimir "Hilo recibió mensaje en puerto: ", PUERTO
74         msg_recibido.tipo <- RECEPCION // Para alinear a la izquierda en la GUI
75         externo <- msg_recibido        // Pasar a la variable global
76         regresa <- 1                    // Activar hilo del pipe escritor
77     Fin Si
78
79     CERRRAR_SOCKET(new_socket)
80     Fin Si
81     Fin Mientras
82
83     CERRRAR_SOCKET(server_fd)
84     TERMINAR_HILO()
85 Fin Funcion
86
87 Funcion hilo_escritor_p2p(arg AS Puntero):
88     Mientras encendido == 1 Hacer:
89         // Si hay una orden de escritura pendiente
90         Si escribe == 1 Entonces
91             escribe <- 0 // Consumir el evento
92             enviar_msj(interfaz.ip_destino, interfaz.puerto_destino, interfaz)
93             Imprimir "Hilo envió mensaje a ", interfaz.puerto_destino
94         Fin Si
95
96         DORMIR_MICROSEGUNDOS(1000) // Evitar que el procesador trabaje al 100% de forma
inútil
97     Fin Mientras
98     TERMINAR_HILO()
99 Fin Funcion
100
101 Funcion hilo_lector_pipe(arg AS Puntero):
102     local AS Referencia a Tuberia <- arg
103     msg_pipe AS Mensaje
104
105     Mientras activo == 1 Hacer:
106         // Espera bloqueante del pipe de la interfaz
107         bytes <- LEER_DE_PIPE(local.B[0], msg_pipe)
108
109         Si bytes > 0 Y activo == 1 Entonces
110             // Caso 1: El usuario cerró la ventana de la GUI
111             Si msg_pipe.id_emisor == -1 Entonces
```

```
112         // Se envía un mensaje de cierre a sí mismo (al servidor local)
113         enviar_msj(DIRECCION, PUERTO, msg_pipe)
114         Romper Bucle
115
116         // Caso 2: El usuario escribió un mensaje ordinario
117         Sino
118             Imprimir "Recibí Mensaje de la GUI..."
119             interfaz <- msg_pipe // Almacenar en buffer global de salida
120             escribe <- 1         // Despertar al escritor P2P
121         Fin Si
122     Fin Si
123 Fin Mientras
124 TERMINAR_HILO()
125 Fin Funcion
126
127 Funcion hilo_escritor_pipe(arg AS Puntero):
128     local AS Referencia a Tuberia <- arg
129
130     Mientras activo == 1 Hacer:
131         // Por lo tanto el servidor P2P nos avisa de mensajes sobre la red
132         Si regresa == 1 Entonces
133             regresa <- 0 // Consumir el evento
134             ESCRIBIR_EN_PIPE(local.A[1], externo) // Enviar directo a la interfaz
135         Fin Si
136
137         DORMIR_MICROSEGUNDOS(1000)
138     Fin Mientras
139     TERMINAR_HILO()
140 Fin Funcion
141
142 Procedimiento enviar_msj(ip_destino AS Cadena, puerto_destino AS Entero, msg AS Mensaje):
143     sock <- CREAM_SOCKET(AF_INET, SOCK_STREAM)
144     Si sock < 0 Entonces Salir
145
146     Destino_Red.puerto <- CONVERTIR_A_FORMATO_RED(puerto_destino)
147     Destino_Red.familia <- AF_INET
148
149     // Convertir IP de texto "127.0.0.1" a binario de red
150     Si CONVERTIR_IP_TEXTO_A_BINARIO(ip_destino, Destino_Red.ip) <= 0 Entonces
151         Cerrar(sock)
152         Salir
153     Fin Si
154
```

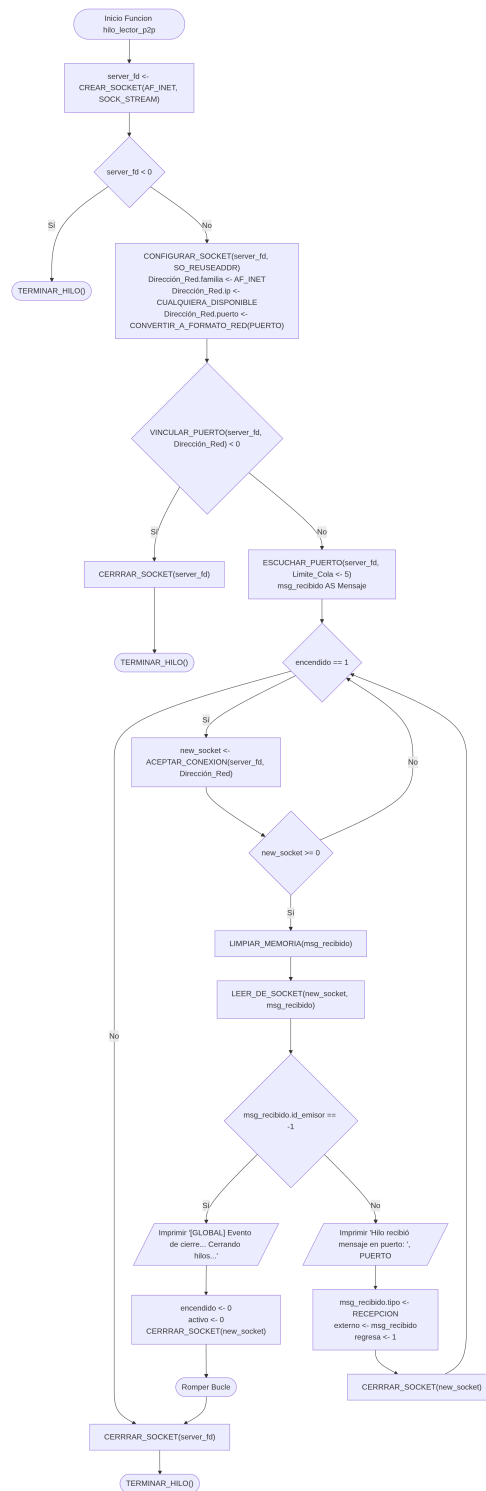
```
155 // Intentar conectar con el nodo remoto (Handshake de 3 vías)
156 Si CONECTAR_A_SOCKET(sock, Destino_Red) >= 0 Entonces
157     ENVIAR_DATOS_SOCKET(sock, msg)
158 Fin Si
159
160 Cerrar(sock)
161 Fin Procedimiento
162
163 Funcion get_time_ms() Devuelve Entero_Grande:
164     tv AS Estructura_Tiempo
165     OBTENER_TIEMPO_DEL_SISTEMA(tv)
166     // Combinar segundos y microsegundos a milisegundos totales
167     Devolver (tv.segundos * 1000) + (tv.microsegundos / 1000)
168 Fin Funcion
169
170
171 Procedimiento formatear_tiempo_str(tiempo_ms AS Entero_Grande, buffer_salida AS
Referencia a Cadena):
172     segundos <- tiempo_ms / 1000
173     milisegundos <- tiempo_ms % 1000
174
175     tm_info AS Estructura_Calendario <- CONVERTIR_A_TIEMPO_LOCAL(segundos)
176
177     // Escribe los milisegundos en formato HH:MM:SS:ms
178     FORMATEAR_CADENA(buffer_salida, "%02d:%02d:%02d:%03d", tm_info.hora, tm_info.minuto,
        tm_info.segundo, milisegundos)
179 Fin Procedimiento
```

#### 4.2.2 Diagrama de flujo (conexion.c)

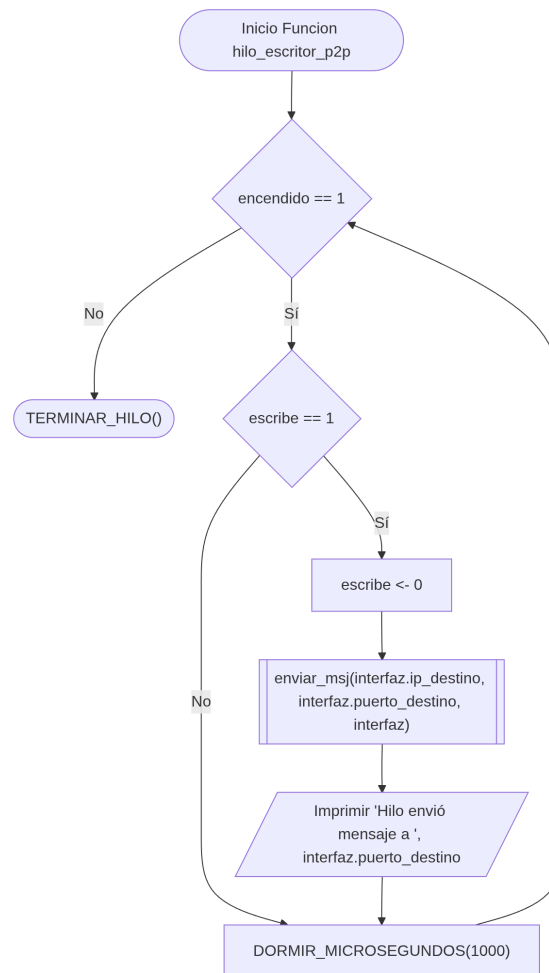
## 1. Procedimiento `conexion()`



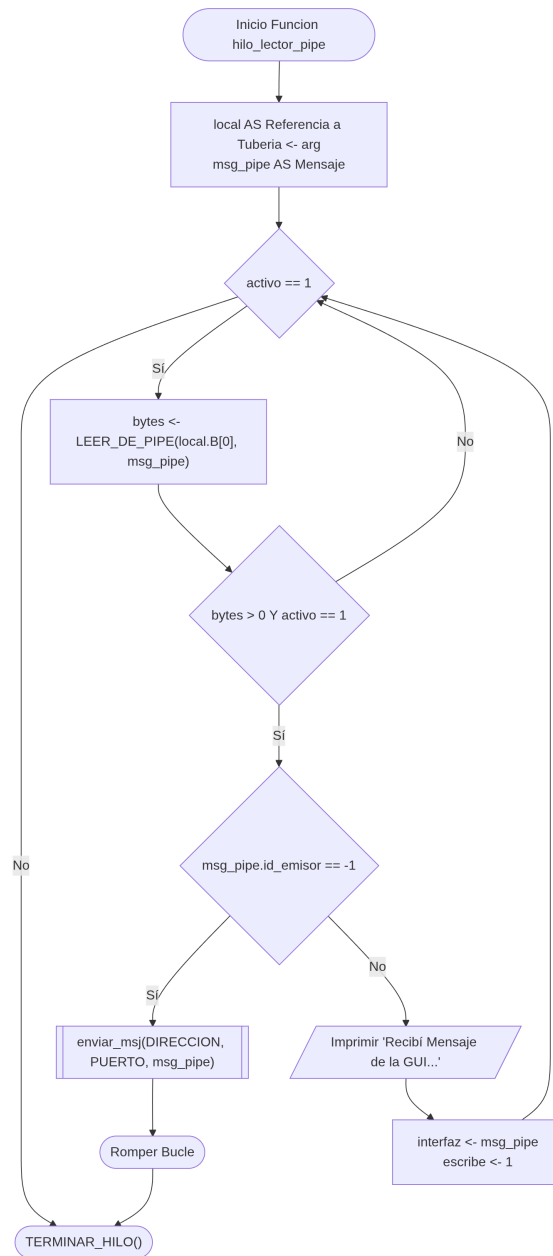
## 2.Función hilo\_lector\_p2p()



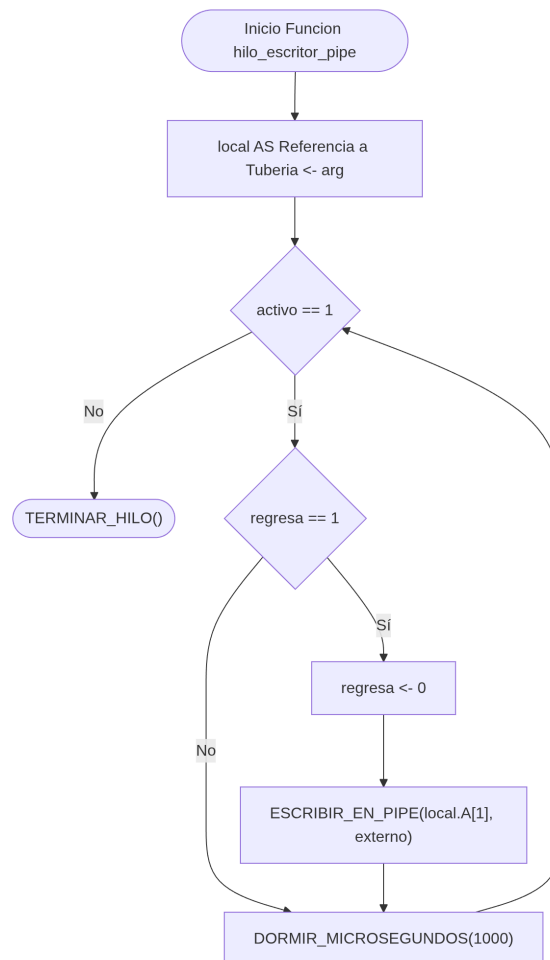
### 3. Función hilo\_escritor\_p2p()



#### 4. Función hilo\_lector\_pipe()

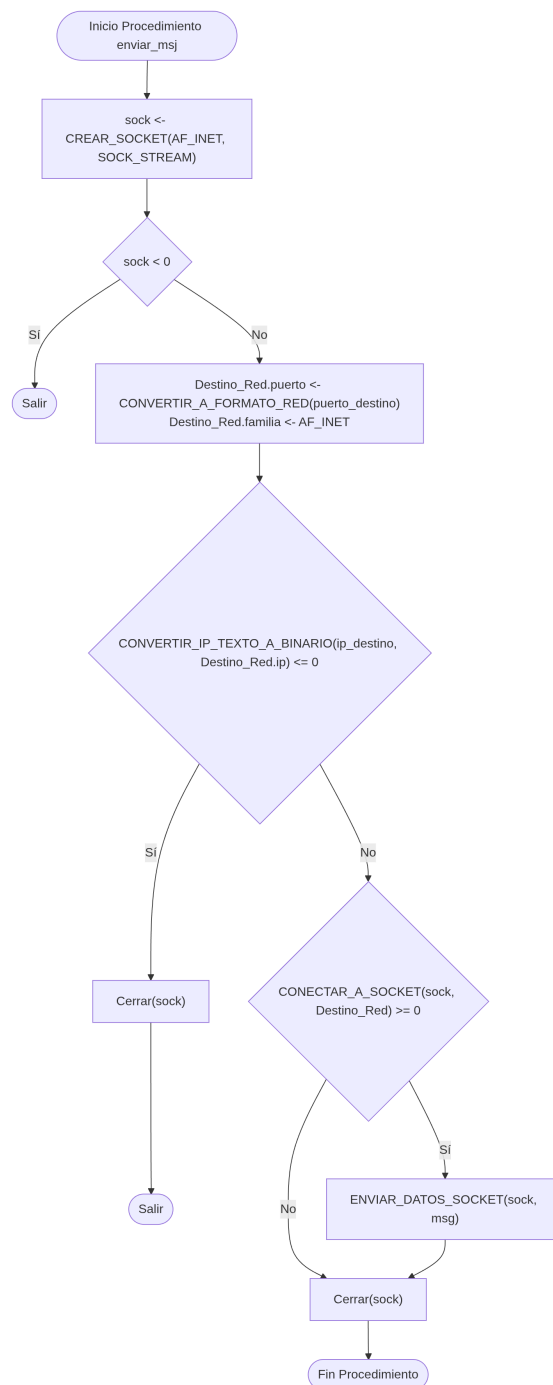


## 5. Función hilo\_escritor\_pipe()

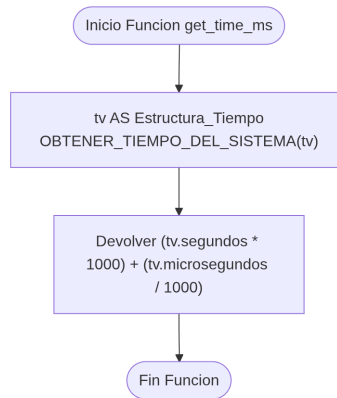




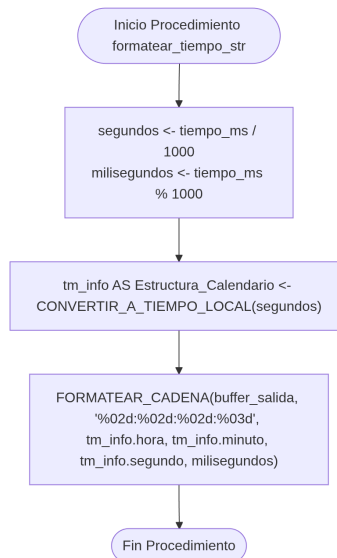
## 6. Procedimiento `enviar_msj()`



## 7. Función `get_time_ms()`



## 8. Procedimiento `formatear_tiempo_str()`



### 4.3 Archivo de Interfaz Gráfica (`interfaz.c`)

Este archivo, de la misma manera que la conexión, se manipula en un proceso independiente (*frontend*) para que no se presenten problemas de congelamiento o bloqueos inesperados en la vista mientras la red opera. Utiliza la biblioteca **GTK+** para el renderizado visual y emplea **Pthreads** para mantener la comunicación asíncrona con el *backend*.

A continuación, se detalla el funcionamiento de los bloques principales y subrutinas que conforman la lógica de la interfaz que se muestra al usuario principal:

- **`interfaz_grafica()`**: Es la subrutina maestra del *frontend*. Inicializa el entorno gráfico de GTK, carga los recursos visuales CSS, y levanta los dos hilos (`gui_lector` y `gui_escritor`). Poste-

riormente, estructura jerárquicamente los contenedores (`GtkBox`, `GtkScrolledWindow`) y widgets (`GtkEntry`, `GtkButton`), cediendo finalmente el control al bucle principal de eventos (`gtk_main()`).

- **Hilos IPC (`hilo_lector_gui` e `hilo_escritor_gui`):** Operan en segundo plano sin interferir con el renderizado de la ventana. El hilo lector extrae los datos entrantes del Pipe A de forma bloqueante y delega la actualización visual al hilo principal de GTK mediante `g_idle_add`. El hilo escritor evalúa activamente la bandera volátil `gui_escribe` para transferir los mensajes del usuario hacia el *backend* a través del Pipe B.
- **Gestión Visual (`añadir_burbuja` y `mostrar_mensaje_recibido`):** Se encargan de instanciar las etiquetas (`GtkLabel`) para el texto del chat. Estas funciones evalúan la procedencia del mensaje (`es_mio`) para anclar la alineación a la derecha (nodo local) o a la izquierda (nodo remoto). Utilizan la sintaxis *Pango Markup* para dar formato a las estampas de tiempo y vinculan los ID del CSS para colorear las burbujas.
- **Persistencia (`cargar_historial`, `guardar_en_historial`, `limpiar_historial_pulsado`):** Implementan un mecanismo local de almacenamiento en el archivo de texto `historial_chat.txt`. Permiten serializar (usando el formato `Origen|Milisegundos|Texto`) y deserializar las cadenas para restaurar conversaciones de sesiones previas al abrir la aplicación, así como truncar el archivo para vaciar el registro.
- **Gestores de Eventos (*Callbacks*):**
  - **`boton_pulsado()`:** Captura el texto del cuadro de entrada al presionar Enter o dar clic en "Enviar", renderiza la burbuja local, lo almacena en disco y levanta la bandera para su envío IPC.
  - **`ventana_destruida()`:** Intercepta el cierre de la ventana por parte del usuario, empaquetando un mensaje especial (`id_emisor = -1`) para forzar el apagado en cascada del *backend* y cerrando el ciclo de GTK.
  - **`scroll_al_final()`:** Ajusta dinámicamente el límite superior del `GtkAdjustment` para que la vista descienda automáticamente al insertar una nueva burbuja de diálogo.
- **`cargar_css()`:** Función integradora que instancia un `GtkCssProvider` para interpretar la hoja de estilos `v-boton.css`, inyectando los gradientes, bordes suavizados y colores distintivos del diseño tipo Messenger directamente al `GdkScreen` de la aplicación.

Listing 3: Archivo del Front-End (conexion P2P)

```
1 #include <gtk/gtk.h>
2 #include <string.h>
3 #include <stdbool.h>
4 #include <unistd.h>
5 #include <pthread.h>
6 #include "header.h"
```

```
7
8 // Variables de control exclusivas para regular el ciclo de vida de los hilos de la GUI
9 volatile int gui_activo = 1;
10 // Bandera volatil para notificar que el usuario presiono enviar y hay datos listos para
    el pipe B
11 volatile int gui_escribe = 0;
12
13 // Buffers globales estructurados para la transferencia asincrona de informacion en la
    GUI
14 Mensaje msg_para_enviar;
15 Mensaje msg_recibido_pipe;
16
17 // Parametros de configuracion de red por defecto para el nodo remoto (amigo)
18 int puerto_amigo = 4001;          // El puerto en el que el otro extremo escucha
    conexiones
19 char dir_amigo[] = "127.0.0.1"; // Direccion IP del nodo remoto (mapeada localmente por
    defecto)
20 int puerto_actual = 4000;          // Puerto asignado a nuestra instancia para desplegarlo
    en la cabecera
21
22 // Componentes globales de GTK requeridos para la manipulacion de la vista desde
    multiples funciones
23 GtkWidget *caja_mensajes;
24 GtkAdjustment *adj_chat;
25 char buffer[TAM_MAX];
26
27 // Puntero de referencia global para interactuar con los descriptores de la tuberia
    principal IPC
28 Tuberia *gui_pipe_ref;
29
30 // Funcion de callback de baja prioridad para ajustar la barra de desplazamiento de
    forma automatica
31 static gboolean scroll_al_final(gpointer data)
32 {
33     // Extraemos el limite superior maximo del area de scroll dinamicamente
34     double max = gtk_adjustment_get_upper(adj_chat);
35     // Obtenemos las dimensiones proporcionales de la pagina visible actual
36     double page = gtk_adjustment_get_page_size(adj_chat);
37     // Movemos el scroll al fondo absoluto restando el tamaño de la pagina visible al
    maximo alcanzado
38     gtk_adjustment_set_value(adj_chat, max - page);
39     // Retornamos FALSE para indicarle al planificador de GLib que destruya este evento
    tras ejecutarse una vez
```

```
40     return FALSE;
41 }
42
43 // Subrutina encargada de construir los elementos visuales (widgets) para representar
    los mensajes del chat
44 static void añadir_burbuja(const char *texto, gboolean es_mio, long long tiempo_ms)
45 {
46     // 1. Instanciamos una etiqueta de GTK para alojar la cadena del mensaje enviado o
    recibido
47     GtkWidget *burbuja = gtk_label_new(texto);
48     // Habilitamos el ajuste de linea automatico si el texto supera las dimensiones
    horizontales maximas
49     gtk_label_set_line_wrap(GTK_LABEL(burbuja), TRUE);
50     // Configuramos que la ruptura de linea se haga por palabras y caracteres de forma
    limpia
51     gtk_label_set_line_wrap_mode(GTK_LABEL(burbuja), PANGO_WRAP_WORD_CHAR);
52     // Establecemos una restriccion de diseño fijando el ancho maximo en 35 caracteres
    por fila
53     gtk_label_set_max_width_chars(GTK_LABEL(burbuja), 35);
54
55     // 2. Declaramos y formateamos la cadena que mostrara la estampa de tiempo
56     char str_tiempo[32];
57     char markup_tiempo[128];
58     formatear_tiempo_str(tiempo_ms, str_tiempo);
59
60     // Formateamos usando sintaxis Pango Markup para reducir la fuente (font='8') y
    conservar un color oscuro
61     sprintf(markup_tiempo, "<span font='8' foreground='#000000'>%s</span>", str_tiempo);
62     GtkWidget *label_tiempo = gtk_label_new(NULL);
63     // Inyectamos el formato enriquecido en la etiqueta correspondiente
64     gtk_label_set_markup(GTK_LABEL(label_tiempo), markup_tiempo);
65
66     // Alineamos verticalmente la etiqueta del tiempo al centro para mantener simetria
    visual
67     gtk_widget_set_valign(label_tiempo, GTK_ALIGN_CENTER);
68
69     // 3. Creamos un contenedor horizontal (HBox) para estructurar la linea del mensaje
    actual
70     GtkWidget *alineacion = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 0);
71     // Asignamos margenes externos a la burbuja para que no quede pegada a las paredes
    ni a otras burbujas
72     gtk_widget_set_margin_start(burbuja, 15);
73     gtk_widget_set_margin_end(burbuja, 15);
```

```
74     gtk_widget_set_margin_top(burbuja, 5);
75     gtk_widget_set_margin_bottom(burbuja, 5);
76
77     // Condicional para ramificar el diseño estilo Messenger dependiendo de la
78     // procedencia del mensaje
79     if (es_mio)
80     {
81         // Si es propio, alineamos todo el contenedor a la extrema derecha
82         gtk_widget_set_halign(alineacion, GTK_ALIGN_END);
83         // Le asignamos el ID CSS "mi-mensaje" definido en v-boton.css (azul con bordes
84         // suavizados)
85         gtk_widget_set_name(burbuja, "mi-mensaje");
86
87         // Estructuramos la distribucion: primero la estampa de tiempo, luego el texto a
88         // la derecha
89         gtk_box_pack_start(GTK_BOX(alineacion), label_tiempo, FALSE, FALSE, 0);
90         gtk_box_pack_start(GTK_BOX(alineacion), burbuja, FALSE, FALSE, 0);
91     }
92     else
93     {
94         // Si proviene de la red, alineamos el contenedor a la extrema izquierda
95         gtk_widget_set_halign(alineacion, GTK_ALIGN_START);
96         // Le asignamos el ID CSS "su-mensaje" definido en v-boton.css (rojo/guinda)
97         gtk_widget_set_name(burbuja, "su-mensaje");
98
99         // Estructuramos la distribucion: primero el texto, luego la estampa de tiempo a
100        // la derecha
101        gtk_box_pack_start(GTK_BOX(alineacion), burbuja, FALSE, FALSE, 0);
102        gtk_box_pack_start(GTK_BOX(alineacion), label_tiempo, FALSE, FALSE, 0);
103    }
104
105    // Insertamos la fila estructurada dentro de la caja contenedora vertical del chat
106    gtk_container_add(GTK_CONTAINER(caja_mensajes), alineacion);
107    // Forzamos el renderizado inmediato del nuevo componente y sus widgets secundarios
108    gtk_widget_show_all(alineacion);
109
110    // Agregamos la rutina de autoscroll al bucle principal de eventos para ejecutarse
111    // en el proximo ciclo de ocio
112    g_idle_add(scroll_al_final, NULL);
113 }
```

```
110 // Funcion para persistir de forma local e incremental la bitacora de conversacion
111 void guardar_en_historial(const char *texto, gboolean es_mio, long long tiempo_ms) {
```

```
112 // Abrimos el archivo en modo "a" (append) para añadir nuevas líneas al final del
documento sin sobrescribir
113 FILE *archivo = fopen("historial_chat.txt", "a");
114 if (archivo != NULL) {
115     // Almacenamos los datos serializados bajo el delimitador de tubería: [Origen][
Milisegundos][Texto]
116     fprintf(archivo, "%d|%lld|%s\n", es_mio ? 1 : 0, tiempo_ms, texto);
117     // Cerramos el flujo del archivo para asegurar la descarga física de datos en el
disco
118     fclose(archivo);
119 }
120 }
121
122 // Callback intermedio invocado de forma segura por el hilo principal de GTK para
procesar buffers remotos
123 static gboolean mostrar_mensaje_recibido(gpointer data)
124 {
125     // Construimos gráficamente la burbuja correspondiente a la izquierda utilizando los
datos del pipe
126     añadir_burbuja(msg_recibido_pipe.texto, FALSE, msg_recibido_pipe.tiempo);
127     // Respalamos de forma permanente el mensaje remoto en el archivo de texto local
guardar_en_historial(msg_recibido_pipe.texto, FALSE, msg_recibido_pipe.tiempo);
128     // Retornamos FALSE para liberar el callback del despachador principal de eventos
return FALSE;
129 }
130
131 // Subrutina invocada al arranque para reconstruir la sesión anterior a partir del
archivo de historial
132 void cargar_historial() {
133     // Abrimos el archivo en modo lectura estricta
134     FILE *archivo = fopen("historial_chat.txt", "r");
135     // Si el archivo no existe en el directorio, abortamos la operación limpiamente sin
lanzar excepciones
136     if (archivo == NULL) return;
137
138     char linea[TAM_MAX + 100];
139     // Leemos secuencialmente línea por línea hasta llegar al final del archivo de texto
while (fgets(linea, sizeof(linea), archivo) != NULL) {
140         // Removemos de forma segura el carácter de salto de línea '\n' remanente de
fgets
141         linea[strcspn(linea, "\n")] = 0;
142
143         int es_mio = 0;
```

```
147     long long tiempo_ms = 0;
148     char texto[TAM_MAX] = {0};
149
150     // Encontramos el primer token de division '|' correspondiente al bit de
procedencia
151     char *ptr1 = strchr(linea, '|');
152     if (ptr1) {
153         *ptr1 = '\0'; // Dividimos la cadena temporalmente introduciendo un
terminador nulo
154         es_mio = atoi(linea); // Convertimos el fragmento ASCII a entero ordinario
(0 o 1)
155
156         // Encontramos el segundo token correspondiente al timestamp en milisegundos
157         char *ptr2 = strchr(ptr1 + 1, '|');
158         if (ptr2) {
159             *ptr2 = '\0'; // Dividimos el fragmento final
160             tiempo_ms = atoll(ptr1 + 1); // Convertimos la cadena de texto a entero
largo de 64 bits (long long)
161             strcpy(texto, ptr2 + 1); // Extraemos el texto integro restante del
mensaje
162
163             // Pintamos la burbuja retrospectivamente preservando si era local o
remota
164             añadir_burbuja(texto, es_mio == 1 ? TRUE : FALSE, tiempo_ms);
165         }
166     }
167 }
168 // Cerramos el descriptor de lectura una vez concluida la carga inicial
169 fclose(archivo);
170 }
171
172 // Callback de evento activado al pulsar el boton "Limpiar"
173 static void limpiar_historial_pulsado(GtkWidget *widget, gpointer data)
174 {
175     // 1. Truncamos y vaciamos por completo el archivo de historial abriendolo en modo
escritura "w"
176     FILE *archivo = fopen("historial_chat.txt", "w");
177     if (archivo != NULL) {
178         fclose(archivo); // Al cerrarse de inmediato, el archivo queda vacio con 0 bytes
179     }
180
181     // 2. Iteramos sobre cada widget hijo contenido en la caja de mensajes y lo
destruimos fisicamente
```



```
182     gtk_container_foreach(GTK_CONTAINER(caja_mensajes), (GtkCallback)gtk_widget_destroy,  
183     NULL);  
184  
185     // Desplegamos bitacora de control en consola  
186     printf("[GUI] Historial limpiado.\n");  
187 }  
188 // Callback invocado cuando el usuario hace clic en el boton Enviar o presiona Enter en  
189 // la entrada de texto  
190 static void boton_pulsado(GtkWidget *widget, gpointer data)  
191 {  
192     GtkEntry *entry = GTK_ENTRY(data);  
193     // Recuperamos el buffer de texto plano de la caja de entrada de GTK  
194     const char *texto = gtk_entry_get_text(entry);  
195  
196     // Proteccion elemental: solo procesamos la entrada si contiene caracteres validos  
197     if (strlen(texto) > 0)  
198     {  
199         // Registramos inmediatamente el tiempo del sistema en milisegundos para  
200         // estampar la operacion  
201         long long mi_tiempo = get_time_ms();  
202         // Agregamos la representacion grafica del mensaje propio apuntando a la derecha  
203         añadir_burbuja(texto, TRUE, mi_tiempo);  
204  
205         // Almacenamos localmente el mensaje en la bitacora persistente  
206         guardar_en_historial(texto, TRUE, mi_tiempo);  
207  
208         // Empaquetamos y serializamos los datos crudos en la estructura Mensaje para su  
209         // transmision IPC  
210         msg_para_enviar.id_emisor = getpid(); // Identificador unico del proceso emisor  
211         // (Frontend)  
212         msg_para_enviar.tipo = ENVIO; // Clasificamos la operacion como un  
213         // mensaje de salida  
214         strncpy(msg_para_enviar.texto, texto, sizeof(msg_para_enviar.texto) - 1);  
215         strncpy(msg_para_enviar.ip_destino, dir_amigo, sizeof(msg_para_enviar.ip_destino)  
216         ) - 1);  
217         msg_para_enviar.puerto_destino = puerto_amigo;  
218         msg_para_enviar.tiempo = mi_tiempo; // Guardamos el timestamp sincronizado  
219         // para la transmision  
220  
221         // Alzamos la bandera volatil para desbloquear la ejecucion del hilo escritor de  
222         // la interfaz  
223         gui_escribe = 1;
```

```
216
217     // Vaciamos la caja de texto para dejarla disponible para el proximo mensaje
218     gtk_entry_set_text(entry, "");
219 }
220 }
221
222 // Manejador del evento de destruccion de la ventana (Cierre de la aplicacion)
223 static void ventana_destruida(GtkWidget *widget, gpointer data)
224 {
225     printf("[GUI] Cerrando ventana. Enviando señal de apagado...\n");
226
227     // Preparamos un mensaje de control especial inyectando -1 en el emisor
228     msg_para_enviar.id_emisor = -1;
229     msg_para_enviar.tipo = SALIDA;
230     strncpy(msg_para_enviar.texto, "Cierre de interfaz.", sizeof(msg_para_enviar.texto)
231 - 1);
232
233     // Forzamos una escritura bloqueante sincrona en el pipe B hacia el backend para
234     ordenar su apagado masivo
235     write(gui_pipe_ref->B[1], &msg_para_enviar, sizeof(Mensaje));
236
237     // Bajamos las banderas globales de control para romper los ciclos infinitos de los
238     hilos remanentes
239     gui_escribe = 0;
240     gui_activo = 0;
241
242     // Rompemos el bucle principal de GTK para devolver la ejecucion al hilo principal
243     gtk_main_quit();
244 }
245
246 // Cargador e interprete del archivo de diseño en cascada CSS para GTK
247 void cargar_css(void)
248 {
249     // Instanciamos un proveedor de estilos CSS para GTK
250     GtkCssProvider *provider = gtk_css_provider_new();
251     // Recuperamos la pantalla por defecto asignada al entorno grafico actual
252     GdkDisplay *display = gdk_display_get_default();
253     GdkScreen *screen = gdk_display_get_default_screen(display);
254
255     // Leemos e interpretamos los selectores declarados en "v-boton.css"
256     gtk_css_provider_load_from_path(provider, "v-boton.css", NULL);
257     // Aplicamos los estilos de forma global a la pantalla con prioridad de aplicacion
258     para sobrescribir temas nativos
```

```
255     gtk_css_provider_add_provider_for_screen(screen, GTK_STYLE_PROVIDER(provider),
GTK_STYLE_PROVIDER_PRIORITY_APPLICATION);
256     // Decrementamos el contador de referencias del proveedor para liberar memoria
limpia
257     g_object_unref(provider);
258 }
259
260 // === HILO 1 DE LA GUI: Lector dedicado en segundo plano para el Pipe A ===
261 void *hilo_lector_gui(void *arg)
262 {
263     Tuberia *local = (Tuberia *)arg;
264     printf("[GUI-HILO-LECTOR] Hilo de lectura de pipe iniciado...\n");
265
266     // Se ejecutara de fondo mientras dure la sesion grafica sin interrumpir los hilos
de render
267     while (gui_activo)
268     {
269         Mensaje temporal;
270         // Operacion bloqueante a nivel de Kernel: espera paquetes provenientes del
backend en el pipe A[0]
271         int bytes = read(local->A[0], &temporal, sizeof(Mensaje));
272
273         // Validamos que se hayan extraido bytes coherentes y la interfaz siga en estado
activo
274         if (bytes > 0 && gui_activo)
275         {
276             // Si el mensaje entrante fue catalogado por el backend como una RECEPCION
de red
277             if (temporal.tipo == RECEPCION)
278             {
279                 printf("Recibi mensaje de la conexion...\n");
280                 // Mapeamos los datos de forma segura en la estructura global dedicada a
la GUI
281                 memcpy(&msg_recibido_pipe, &temporal, sizeof(Mensaje));
282                 // Delegamos de forma asincrona e indolora la insercion grafica en el
hilo principal de GTK
283                 g_idle_add(mostrar_mensaje_recibido, NULL);
284             }
285         }
286         else
287         {
288             // Si el descriptor es cerrado o da error, rompemos el ciclo de ejecucion de
forma inmediata
```

```
289         break;
290     }
291 }
292 printf("[GUI-HILO-LECTOR] Saliendo del hilo lector de la interfaz.\n");
293 // Terminamos de forma limpia e independiente la existencia de este hilo
294 pthread_exit(NULL);
295 }
296
297 // === HILO 2 DE LA GUI: Escritor dedicado en segundo plano para canalizar en el Pipe B
298 // ===
299 void *hilo_escritor_gui(void *arg)
300 {
301     Tuberia *local = (Tuberia *)arg;
302     printf("[GUI-HILO-ESCRITOR] Hilo de escritura de pipe iniciado...\n");
303
304     while (gui_activo)
305     {
306         // Evaluamos de forma constante mediante sondeo (polling) el estado de la
307         // bandera de salida
308         if (gui_escribe == 1)
309         {
310             // Despachamos la estructura serializada por el pipe B para que el modulo de
311             // red lo procese
312             write(local->B[1], &msg_para_enviar, sizeof(Mensaje));
313             // Apagamos la señal inmediatamente para evitar reenvios duplicados
314             gui_escribe = 0;
315         }
316         // Dormimos el hilo durante 1 milisegundo para evitar que el bucle vacio consuma
317         // el 100% de un nucleo del CPU
318         usleep(1000);
319     }
320     printf("[GUI-HILO-ESCRITOR] Saliendo del hilo escritor de la interfaz.\n");
321     pthread_exit(NULL);
322 }
323
324 // Inicializador maestro de la interfaz grafica de usuario
325 void interfaz_grafica(int argc, char *argv[], Tuberia *padre)
326 {
327     // Descriptores internos para almacenar la configuracion geometrica y jerarquica de
328     // los componentes de GTK
329     GtkWidget *window, *box, *entry, *scrolled_window, *hbox, *button, *header_box, *
330     info_contacto;
331     pthread_t gui_lector, gui_escritor;
```

```
326
327 // Resguardamos localmente la direccion de memoria del canal de pipes cruzados
328 gui_pipe_ref = padre;
329
330 // Inicializamos el entorno basico de GTK abstrayendo los argumentos de linea de
331 comandos
332 gtk_init(&argc, &argv);
333 // Inyectamos las hojas de estilo personalizadas estilo Messenger retro-futurista
334 cargar_css();
335
336 // --- CONCURRENCIA: LEVANTAR LOS DOS HILOS INDEPENDIENTES DE LA GUI ---
337 pthread_create(&gui_lector, NULL, hilo_lector_gui, (void *)gui_pipe_ref);
338 pthread_create(&gui_escritor, NULL, hilo_escritor_gui, (void *)gui_pipe_ref);
339
340 // Construimos la ventana principal y definimos sus atributos generales
341 window = gtk_window_new(GTK_WINDOW_TOPLEVEL);
342 gtk_window_set_title(GTK_WINDOW(window), "MSN P2P - INTERFAZ");
343 gtk_window_set_default_size(GTK_WINDOW(window), 450, 650);
344
345 // Vinculamos la señal de destruccion fisica de la ventana al callback de control
346 ventana_destruida
347 g_signal_connect(window, "destroy", G_CALLBACK(ventana_destruida), NULL);
348
349 // Contenedor base vertical que apilara la cabecera, la caja de texto y la seccion
350 de entrada
351 box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 0);
352 gtk_container_add(GTK_CONTAINER(window), box);
353
354 // --- DISEÑO DE LA CABECERA (Header estilo MSN) ---
355 header_box = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 10);
356 gtk_widget_set_name(header_box, "header-messenger");
357 gtk_box_pack_start(GTK_BOX(box), header_box, FALSE, FALSE, 0);
358 info_contacto = gtk_label_new(NULL);
359
360 // Redactamos el banner informativo inyectando etiquetas con formato enriquecido
361 mediante snprintf
362 snprintf(buffer, sizeof(buffer),
363          "<span font='11' weight='bold' foreground='#003399'>MESSENGER P2P</span>\n"
364          "<span font='9' foreground='#666'>Escucha: %d - Conectado</span>",
365          puerto_actual);
366
367 gtk_label_set_markup(GTK_LABEL(info_contacto), buffer);
368 gtk_box_pack_start(GTK_BOX(header_box), info_contacto, FALSE, FALSE, 15);
```

```
365
366 // --- COMPONENTE COMPLEMENTARIO: BOTON DE LIMPIEZA DE BITACORA ---
367 GtkWidget *btn_limpiar = gtk_button_new_with_label("Limpiar");
368 g_signal_connect(btn_limpiar, "clicked", G_CALLBACK(limpiar_historial_pulsado), NULL
);
369 // Empaquetamos el boton al final (pack_end) para arrastrarlo al extremo derecho de
la cabecera
370 gtk_box_pack_end(GTK_BOX(header_box), btn_limpiar, FALSE, FALSE, 15);
371
372 // --- COMPONENTE DE DESPLAZAMIENTO (Scrolled Window) ---
373 scrolled_window = gtk_scrolled_window_new(NULL, NULL);
374 gtk_box_pack_start(GTK_BOX(box), scrolled_window, TRUE, TRUE, 0);
375 // Extraemos la configuracion geometrica del ajuste vertical para controlar la
posicion de la barra
376 adj_chat = gtk_scrolled_window_get_vadjustment(GTK_SCROLLED_WINDOW(scrolled_window))
;
377
378 // Instanciamos el contenedor interno vertical donde se iran apilando dinamicamente
las lineas del chat
379 caja_mensajes = gtk_box_new(GTK_ORIENTATION_VERTICAL, 0);
380 gtk_widget_set_valign(caja_mensajes, GTK_ALIGN_END); // Anclamos el crecimiento
hacia abajo
381 gtk_container_add(GTK_CONTAINER(scrolled_window), caja_mensajes);
382
383 // --- AREA INFERIOR DE CAPTURA (HBox de entrada y transmision) ---
384 hbox = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 5);
385 gtk_widget_set_margin_start(hbox, 10);
386 gtk_widget_set_margin_end(hbox, 10);
387 gtk_widget_set_margin_top(hbox, 10);
388 gtk_widget_set_margin_bottom(hbox, 10);
389 gtk_box_pack_start(GTK_BOX(box), hbox, FALSE, FALSE, 0);
390
391 // Instanciamos el cuadro de entrada de texto plano (GtkEntry)
392 entry = gtk_entry_new();
393 gtk_entry_set_placeholder_text(GTK_ENTRY(entry), "Escribe un mensaje...");
394
395 // Conectamos el evento "activate" (presionar Enter dentro del cuadro) al callback
boton_pulsado
396 g_signal_connect(entry, "activate", G_CALLBACK(boton_pulsado), entry);
397 gtk_box_pack_start(GTK_BOX(hbox), entry, TRUE, TRUE, 0);
398
399 // Instanciamos el boton fisico de Enviar
400 button = gtk_button_new_with_label("Enviar");
```

```
401 // Conectamos el click del raton al mismo callback compartiendo el puntero de la
    caja de texto
402 g_signal_connect(button, "clicked", G_CALLBACK(boton_pulsado), entry);
403 gtk_box_pack_start(GTK_BOX(hbox), button, FALSE, FALSE, 0);
404
405 // Reconstruimos la conversacion anterior leyendo de disco antes de levantar la
    interfaz visible
406 cargar_historial();
407
408 // Mostramos la ventana y ordenamos la cascada jerarquica de widgets internos
409 gtk_widget_show_all(window);
410
411 // Bloqueamos el hilo principal cediendo el control total al despachador de eventos
    de GTK (Bucle principal)
412 gtk_main();
413
414 // Sincronizacion final: Al romperse el bucle, esperamos la terminacion e
    incorporacion de los hilos asincronos
415 pthread_join(gui_lector, NULL);
416 pthread_join(gui_escritor, NULL);
417 }
```

### 4.3.1 Pseudocódigo (interfaz.c)

```
1 // Banderas de control de ciclos de vida de la interfaz
2 Volatil Entero gui_activo <- 1
3 Volatil Entero gui_escribe <- 0
4
5 // Estructuras de datos para comunicación asíncrona
6 Mensaje msg_para_enviar // Datos empaquetados listos para salir por el Pipe B
7 Mensaje msg_recibido_pipe // Datos extraídos del Pipe A listos para graficarse
8
9 // Configuración de red del nodo remoto (amigo) y local
10 Entero puerto_amigo <- 4001
11 Cadena dir_amigo <- "127.0.0.1"
12 Entero puerto_actual <- 4000
13
14 // Componentes globales de GTK (Referencias de la Vista)
15 Componente caja_mensajes // Contenedor vertical de las burbujas de chat
16 Componente adj_chat // Control del scroll de la ventana de chat
17 Cadena buffer[TAM_MAX] // Buffer de texto genérico
18
19 // Puntero de referencia a la tubería IPC principal
```

```
20 Tuberia *gui_pipe_ref
21
22 Procedimiento interfaz_grafica(argc, argv[], padre AS Referencia a Tuberia):
23     gui_pipe_ref <- padre
24
25     INICIALIZAR_ENTORNO_GTK(argc, argv)
26     cargar_css() // Cargar estilos visuales desde "v-boton.css"
27
28     // --- CONCURRENCIA: Lanzar hilos de comunicación con el Backend ---
29     CREAR_HILO(gui_lector, hilo_lector_gui, gui_pipe_ref)
30     CREAR_HILO(gui_escritor, hilo_escritor_gui, gui_pipe_ref)
31
32     // --- CONSTRUCCIÓN DE LA VENTANA PRINCIPAL ---
33     ventana <- CREAM_VENTANA_TOPLEVEL()
34     CONFIGURAR_TITULO(ventana, "MSN P2P - INTERFAZ")
35     CONFIGURAR_TAMAÑO(ventana, ancho <- 450, alto <- 650)
36     CONECTAR_SEÑAL(ventana, "destroy", ventana_destruida)
37
38     caja_base <- CREAM_CAJA_VERTICAL(espaciado <- 0)
39     AGREGAR_A_CONTENEDOR(ventana, caja_base)
40
41     // --- DISEÑO DE LA CABECERA (Estilo MSN) ---
42     cabecera <- CREAM_CAJA_HORIZONTAL(espaciado <- 10)
43     ASIGNAR_NOMBRE_CSS(cabecera, "header-messenger")
44     EMPAQUETAR_INICIO(caja_base, cabecera, expandir <- Falso)
45
46     label_info <- CREAM_ETIQUETA()
47     FORMATEAR_CADENA(buffer, "<b>MESSENGER P2P</b>\nEscucha: %d - Conectado",
48     puerto_actual)
49     ASIGNAR_TEXTO_MARKUP(label_info, buffer)
50     EMPAQUETAR_INICIO(cabecera, label_info, expandir <- Falso, margen <- 15)
51
52     btn_limpiar <- CREAM_BOTON_CON_TEXTO("Limpiar")
53     CONECTAR_SEÑAL(btn_limpiar, "clicked", limpiar_historial_pulsado)
54     EMPAQUETAR_FINAL(cabecera, btn_limpiar, expandir <- Falso, margen <- 15)
55
56     // --- ÁREA DE CHAT CON SCROLL ---
57     ventana_scroll <- CREAM_VENTANA_CON_SCROLL(Nulo, Nulo)
58     EMPAQUETAR_INICIO(caja_base, ventana_scroll, expandir <- Verdadero)
59     adj_chat <- OBTENER_AJUSTE_VERTICAL(ventana_scroll)
60
61     caja_mensajes <- CREAM_CAJA_VERTICAL(espaciado <- 0)
62     ALINEAR_VERTICAL_AL_FONDO(caja_mensajes) // Crecimiento natural hacia abajo
```



```
62  AGREGAR_A_CONTENEDOR(ventana_scroll, caja_mensajes)
63
64  // --- ÁREA INFERIOR DE ENTRADA DE TEXTO ---
65  caja_entrada <- CREAM_CAJA_HORIZONTAL(espaciado <- 5)
66  CONFIGURAR_MARGINS(caja_entrada, izq <- 10, der <- 10, sup <- 10, inf <- 10)
67  EMPAQUETAR_INICIO(caja_base, caja_entrada, expandir <- Falso)
68
69  campo_texto <- CREAM_CAMPO_ENTRADA_TEXTO()
70  CONFIGURAR_TEXTO_FONDO(campo_texto, "Escribe un mensaje...")
71  CONECTAR_SEÑAL(campo_texto, "activate", boton_pulsado, campo_texto) // Al presionar
Enter
72  EMPAQUETAR_INICIO(caja_entrada, campo_texto, expandir <- Verdadero)
73
74  btn_enviar <- CREAM_BOTON_CON_TEXTO("Enviar")
75  CONECTAR_SEÑAL(btn_enviar, "clicked", boton_pulsado, campo_texto) // Al realizar
clic
76  EMPAQUETAR_INICIO(caja_entrada, btn_enviar, expandir <- Falso)
77
78  // --- ARRANCAR SESIÓN ---
79  cargar_historial() // Cargar mensajes previos desde el disco duro
80  MOSTRAR_TODO(ventana) // Renderizar componentes gráficos
81
82  EJECUTAR_BUCLE_PRINCIPAL_GTK() // Cede el control total a GTK (Bloqueante)
83
84  // --- LIMPIEZA POST-CIERRE ---
85  HILO_ESPERAR(gui_lector)
86  HILO_ESPERAR(gui_escritor)
87  Fin Procedimiento
88
89  Funcion hilo_lector_gui(arg AS Puntero) Devuelve Puntero:
90  local AS Referencia a Tuberia <- arg
91  Imprimir "[GUI-HILO-LECTOR] Hilo de lectura de pipe iniciado..."
92
93  Mientras gui_activo == 1 Hacer:
94  temporal AS Mensaje
95  // Operación bloqueante del Sistema Operativo en Pipe A[0]
96  bytes <- LEER_DE_PIPE(local.A[0], temporal)
97
98  Si bytes > 0 Y gui_activo == 1 Entonces
99  // Si el mensaje es una recepción legítima del exterior
100  Si temporal.tipo == RECEPCION Entonces
101  Imprimir "Recibí mensaje de la conexión..."
102  COPIAR_MEMORIA(msg_recibido_pipe, temporal)
```

```
103         // Delegar el renderizado al hilo principal de la GUI cuando esté libre
104         AGREGAR_A_CICLO_OCIO_GLIB(mostrar_mensaje_recibido)
105         Fin Si
106     Sino
107         Romper Bucle // Error o pipe roto, apagar hilo
108     Fin Si
109 Fin Mientras
110
111 Imprimir "[GUI-HILO-LECTOR] Saliendo del hilo lector."
112 TERMINAR_HILO()
113 Fin Funcion
114
115 Funcion hilo_escritor_gui(arg AS Puntero) Devuelve Puntero:
116     local AS Referencia a Tuberia <- arg
117     Imprimir "[GUI-HILO-ESCRITOR] Hilo de escritura de pipe iniciado..."
118
119     Mientras gui_activo == 1 Hacer:
120         // Sondeo continuo de la bandera controlada por el botón de Enviar
121         Si gui_escribe == 1 Entonces
122             ESCRIBIR_EN_PIPE(local^.B[1], msg_para_enviar)
123             gui_escribe <- 0 // Consumir el evento inmediatamente
124         Fin Si
125
126         DORMIR_MICROSEGUNDOS(1000) // Evitar saturación del procesador
127     Fin Mientras
128
129     Imprimir "[GUI-HILO-ESCRITOR] Saliendo del hilo escritor."
130     TERMINAR_HILO()
131 Fin Funcion
132
133 Procedimiento añadir_burbuja(texto AS Cadena, es_mio AS Booleano, tiempo_ms AS
134     Entero_Grande):
135     burbuja <- CREAR_ETIQUETA_TEXTO(texto)
136     CONFIGURAR_AJUSTE_LINEA(burbuja, Verdadero)
137     CONFIGURAR_ANCHO_MAXIMO_CARACTERES(burbuja, 35)
138
139     // Formatear estampa de tiempo usando Pango Markup (fuente pequeña)
140     str_tiempo AS Cadena
141     markup_tiempo AS Cadena
142     formatear_tiempo_str(tiempo_ms, str_tiempo)
143     FORMATEAR_CADENA(markup_tiempo, "<span font='8' foreground='#000000'>%s</span>",
```

```
144 label_tiempo <- CREAM_ETIQUETA_TEXTO_VACIA()
145 ASIGNAR_TEXTO_MARKUP(label_tiempo, markup_tiempo)
146 ALINEAR_VERTICAL_AL_CENTRO(label_tiempo)
147
148 fila_alineacion <- CREAM_CAJA_HORIZONTAL(espaciado <- 0)
149 CONFIGURAR_MARGINS(burbuja, izq <- 15, der <- 15, sup <- 5, inf <- 5)
150
151 Si es_mio == Verdadero Entonces
152     ALINEAR_HORIZONTAL_A_LA_DERECHA(fila_alineacion)
153     ASIGNAR_ID_CSS(burbuja, "mi-mensaje") // Color azul en CSS
154     // Estructura: [Tiempo] [Texto]
155     EMPAQUETAR_INICIO(fila_alineacion, label_tiempo, expandir <- Falso)
156     EMPAQUETAR_INICIO(fila_alineacion, burbuja, expandir <- Falso)
157 Sino
158     ALINEAR_HORIZONTAL_A_LA_IZQUIERDA(fila_alineacion)
159     ASIGNAR_ID_CSS(burbuja, "su-mensaje") // Color guinda en CSS
160     // Estructura: [Texto] [Tiempo]
161     EMPAQUETAR_INICIO(fila_alineacion, burbuja, expandir <- Falso)
162     EMPAQUETAR_INICIO(fila_alineacion, label_tiempo, expandir <- Falso)
163 Fin Si
164
165 AGREGAR_A_CONTENEDOR(caja_mensajes, fila_alineacion)
166 MOSTRAR_ELEMENTO_Y_HIJOS(fila_alineacion)
167
168 // Programar auto-scroll automático
169 AGREGAR_A_CICLO_OCIO_GLIB(scroll_al_final)
170 Fin Procedimiento
171
172 Procedimiento boton_pulsado(widget, data):
173     entry_widget <- Convertir_A_GtkEntry(data)
174     texto AS Cadena <- OBTENER_TEXTO_DE_CAMPO(entry_widget)
175
176 Si LONGITUD_CADENA(texto) > 0 Entonces
177     mi_tiempo <- get_time_ms()
178
179     // Pintar en pantalla y guardar localmente en el archivo txt
180     añadir_burbuja(texto, TRUE, mi_tiempo)
181     guardar_en_historial(texto, TRUE, mi_tiempo)
182
183     // Serializar estructura para mandar al Backend por el Pipe B
184     msg_para_enviar.id_emisor <- OBTENER_PID_PROCESO_ACTUAL()
185     msg_para_enviar.tipo <- ENVIO
186     COPIAR_CADENA(msg_para_enviar.texto, texto)
```

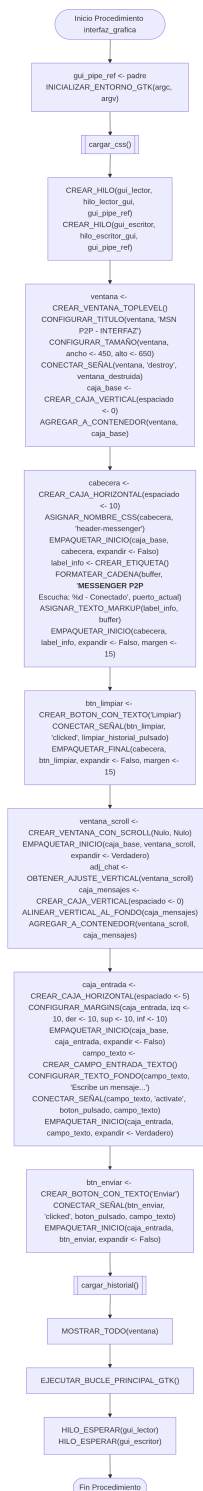
```
187     COPIAR_CADENA(msg_para_enviar.ip_destino, dir_amigo)
188     msg_para_enviar.puerto_destino <- puerto_amigo
189     msg_para_enviar.tiempo <- mi_tiempo
190
191     gui_escribe <- 1 // Despertar al hilo escritor de la GUI
192     VACIAR_CAMPO_TEXTO(entry_widget) // Dejar limpio para el siguiente texto
193     Fin Si
194 Fin Procedimiento
195
196 Procedimiento ventana_destruida(widget, data):
197     Imprimir "[GUI] Cerrando ventana. Enviando señal de apagado..."
198
199     // Envasar mensaje de control con ID de apagado global (-1)
200     msg_para_enviar.id_emisor <- -1
201     msg_para_enviar.tipo <- SALIDA
202     COPIAR_CADENA(msg_para_enviar.texto, "Cierre de interfaz.")
203
204     // Escritura forzada y síncrona en el Pipe B para apagar el backend antes de morir
205     ESCRIBIR_EN_PIPE(gui_pipe_ref^.B[1], msg_para_enviar)
206
207     gui_escribe <- 0
208     gui_activo <- 0 // Forzar salida de los bucles de los hilos remanentes
209
210     TERMINAR_BUCLE_PRINCIPAL_GTK()
211 Fin Procedimiento
212
213 Procedimiento guardar_en_historial(texto AS Cadena, es_mio AS Booleano, tiempo_ms AS
    Entero_Grande):
214     archivo <- ABRIR_ARCHIVO("historial_chat.txt", Modo <- "Añadir")
215     Si archivo != Nulo Entonces
216         Entero_Origen <- es_mio ? 1 : 0
217         IMPRIMIR_EN_ARCHIVO(archivo, "%d|%lld|%s\n", Entero_Origen, tiempo_ms, texto)
218         CERRAR_ARCHIVO(archivo)
219     Fin Si
220 Fin Procedimiento
221
222
223 Procedimiento cargar_historial():
224     archivo <- ABRIR_ARCHIVO("historial_chat.txt", Modo <- "Lectura")
225     Si archivo == Nulo Entonces Salir
226
227     linea AS Cadena
228     Mientras LEER_LINEA_DE_ARCHIVO(linea, archivo) != Nulo Hacer:
```

```
229     ELIMINAR_SALTO_LINEA_FINAL(linea)
230
231     // Tokenización manual usando el delimitador '|'
232     ptr1 <- ENCONTRAR_CARACTER(linea, '|')
233     Si ptr1 != Nulo Entonces
234         REEMPLAZAR_CON_NULO(ptr1)
235         es_mio <- CONVERTIR_A_ENTERO(linea)
236
237     ptr2 <- ENCONTRAR_CARACTER(ptr1 + 1, '|')
238     Si ptr2 != Nulo Entonces
239         REEMPLAZAR_CON_NULO(ptr2)
240         tiempo_ms <- CONVERTIR_A_ENTERO_LONG_LONG(ptr1 + 1)
241         texto <- EXTRAER_CADENA(ptr2 + 1)
242
243         // Regenerar burbujas del pasado de forma retrospectiva
244         añadir_burbuja(texto, es_mio == 1 ? Verdadero : Falso, tiempo_ms)
245     Fin Si
246 Fin Si
247 Fin Mientras
248 CERRAR_ARCHIVO(archivo)
249 Fin Procedimiento
250
251
252 Procedimiento limpiar_historial_pulsado(widget, data):
253     // Truncar el archivo a 0 bytes
254     archivo <- ABRIR_ARCHIVO("historial_chat.txt", Modo <- "Escritura_Limpia")
255     Si archivo != Nulo Entonces CERRAR_ARCHIVO(archivo)
256
257     // Destruir todos los objetos gráficos burbuja dentro del contenedor
258     PARA_CADA_HIJO_EN_CONTENEDOR(caja_mensajes, DESTRUIR_WIDGET)
259
260     Imprimir "[GUI] Historial limpiado."
261 Fin Procedimiento
262
263 Funcion scroll_al_final(data AS Puntero) Devuelve Booleano:
264     max_scroll <- OBTENER_VALOR_SUPERIOR_MAXIMO(adj_chat)
265     tamaño_pagina <- OBTENER_TAMAÑO_PAGINA_VISIBLE(adj_chat)
266     ASIGNAR_VALOR_SCROLL(adj_chat, max_scroll - tamaño_pagina)
267     Devolver Falso // Auto-destruir callback de eventos diferidos
268 Fin Funcion
269
270 Funcion mostrar_mensaje_recibido(data AS Puntero) Devuelve Booleano:
271     añadir_burbuja(msg_recibido_pipe.texto, Falso, msg_recibido_pipe.tiempo)
```

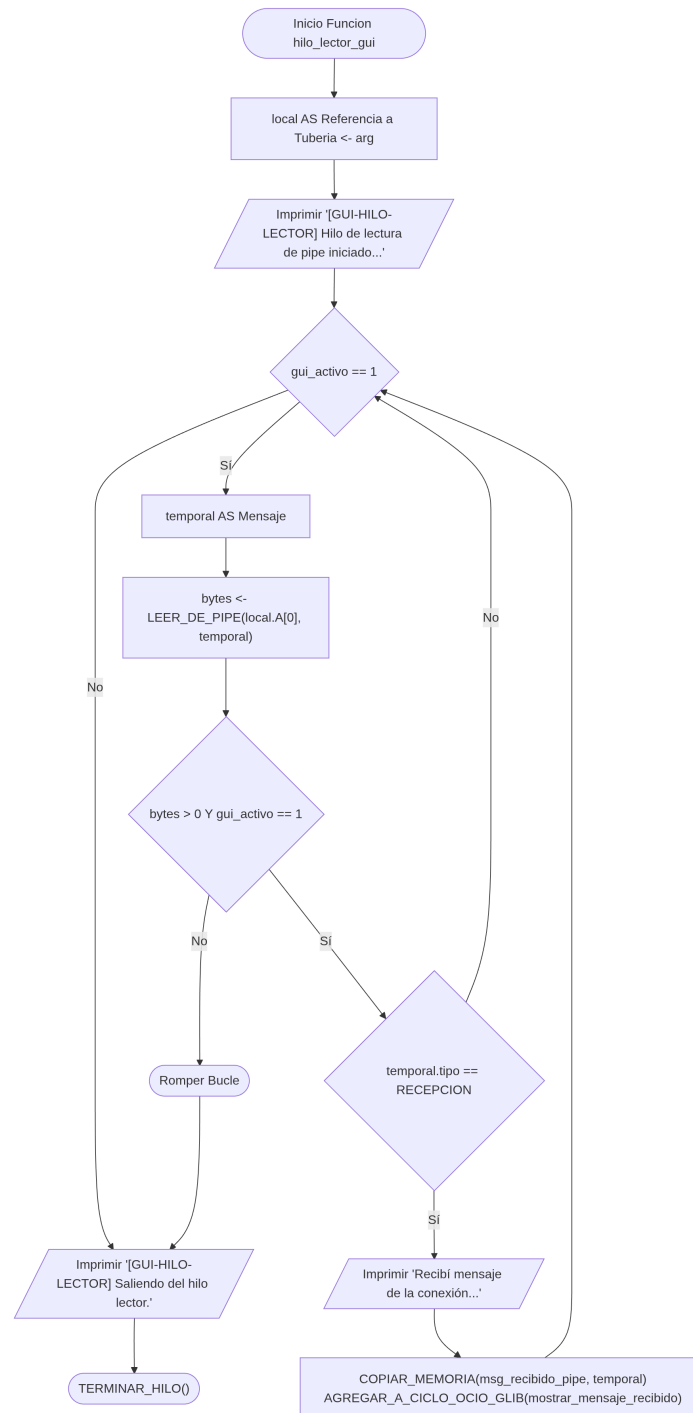
```
272     guardar_en_historial(msg_recibido_pipe.texto, Falso, msg_recibido_pipe.tiempo)
273     Devolver Falso
274 Fin Funcion
275
276 Procedimiento cargar_css():
277     proveedor <- CREAM_PROVEEDOR_CSS()
278     pantalla <- OBTENER_PANTALLA_GRAFICA_SISTEMA()
279     INTERPRETAR_ARCHIVO_CSS_DESDE_RUTA(proveedor, "v-boton.css")
280     INYECTAR_ESTILOS_A_PANTALLA(pantalla, proveedor, PRIORIDAD_APLICACION)
281 Fin Procedimiento
```

#### 4.3.2 Diagrama de flujo (interfaz.c)

## 1. Procedimiento interfaz\_grafica()

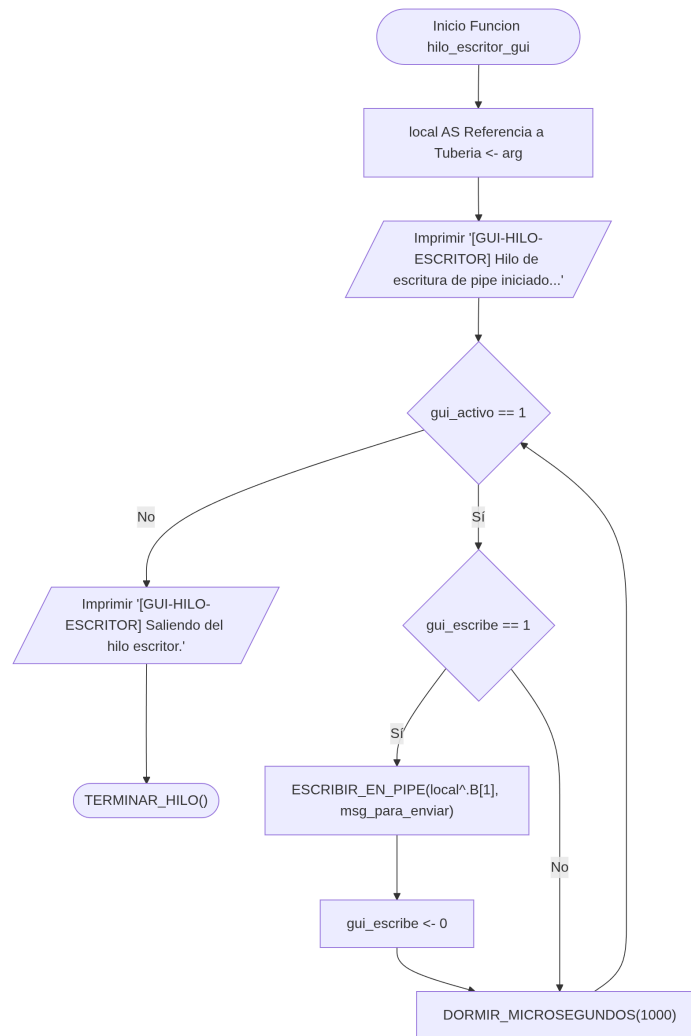


## 2. Función hilo\_lector\_gui()

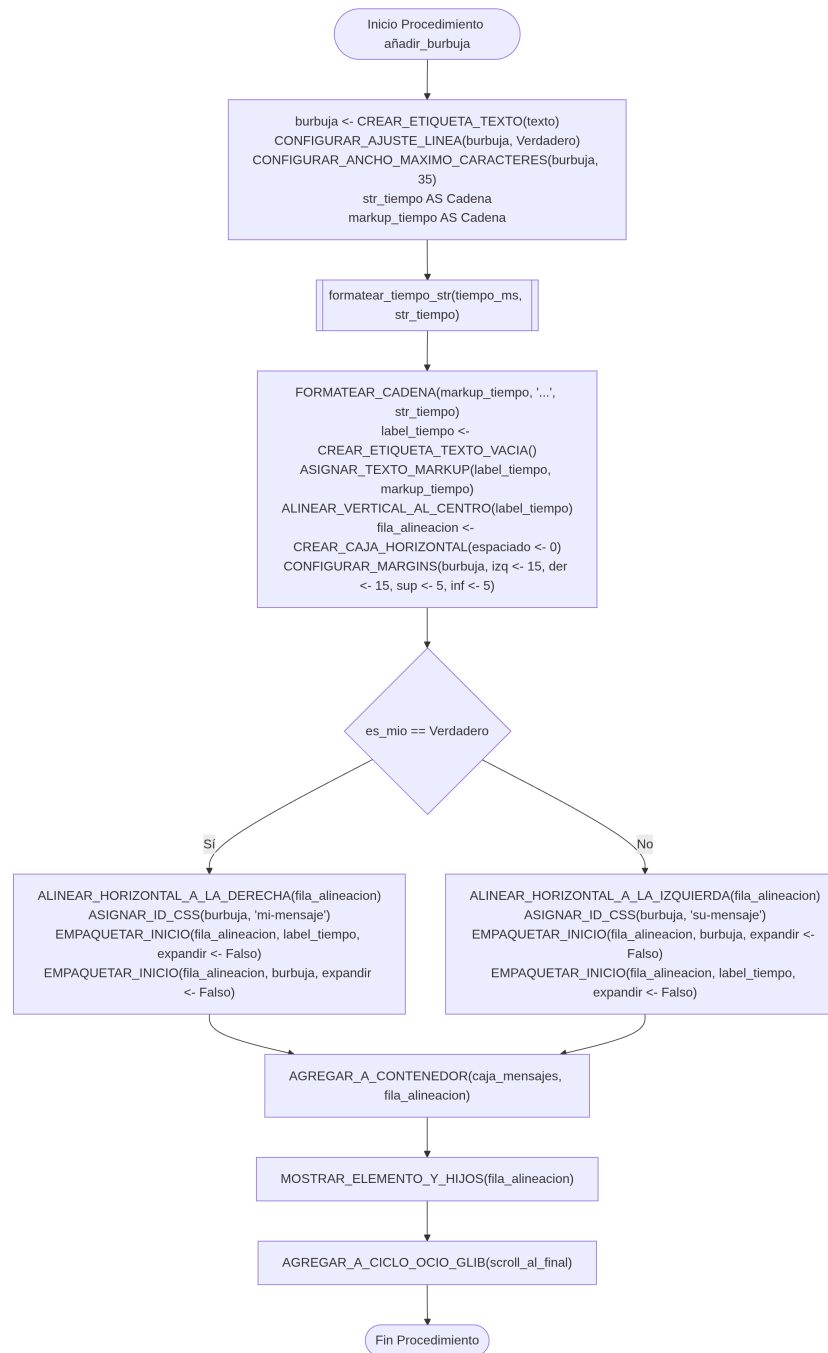




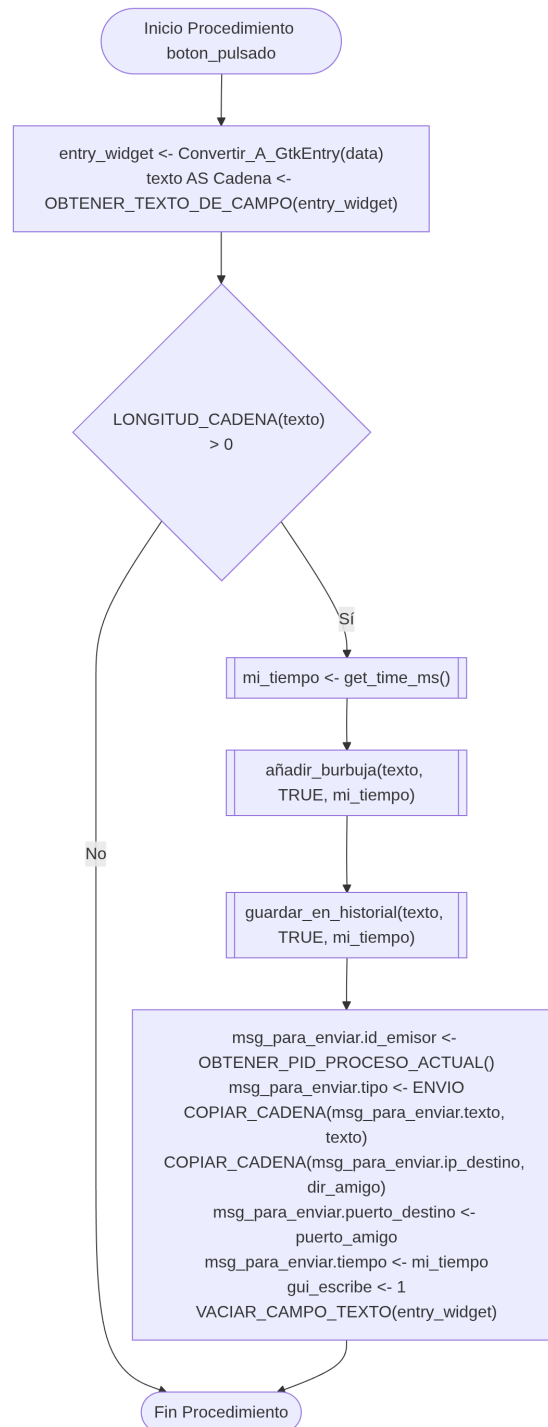
### 3. Función hilo\_escritor\_gui()



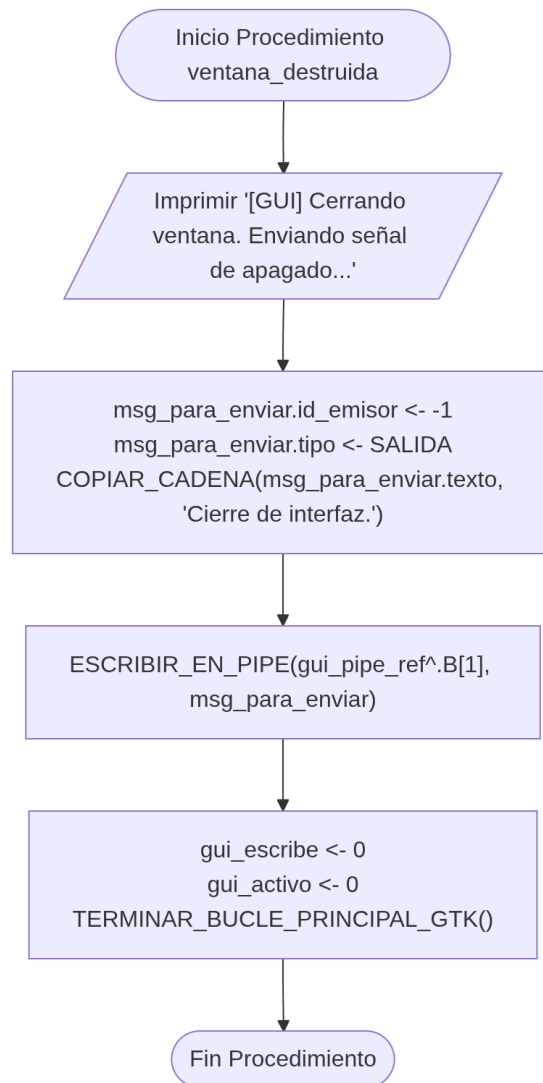
#### 4. Procedimiento añadir\_burbuja()



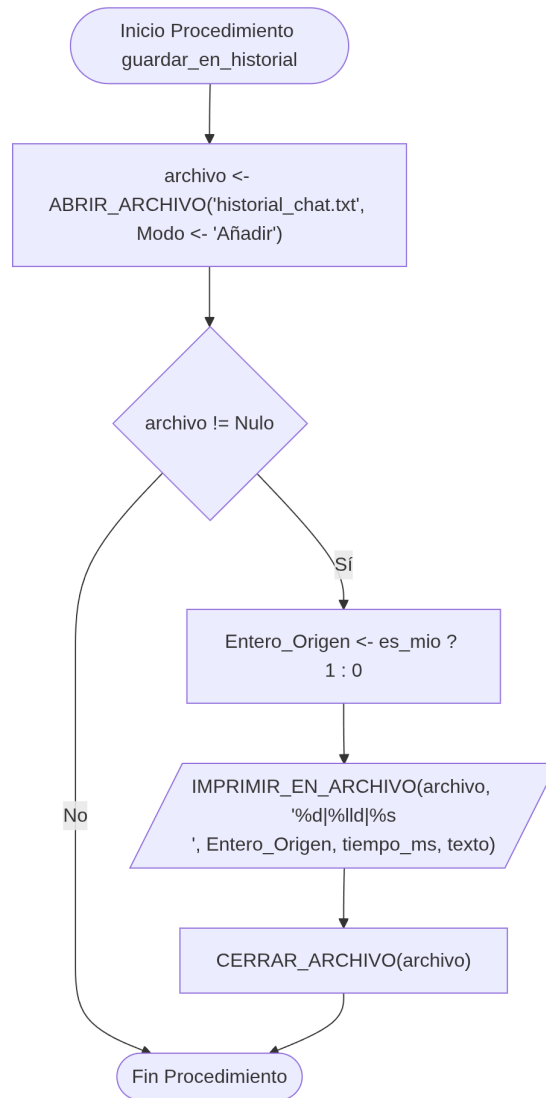
## 5. Procedimiento boton\_pulsado()



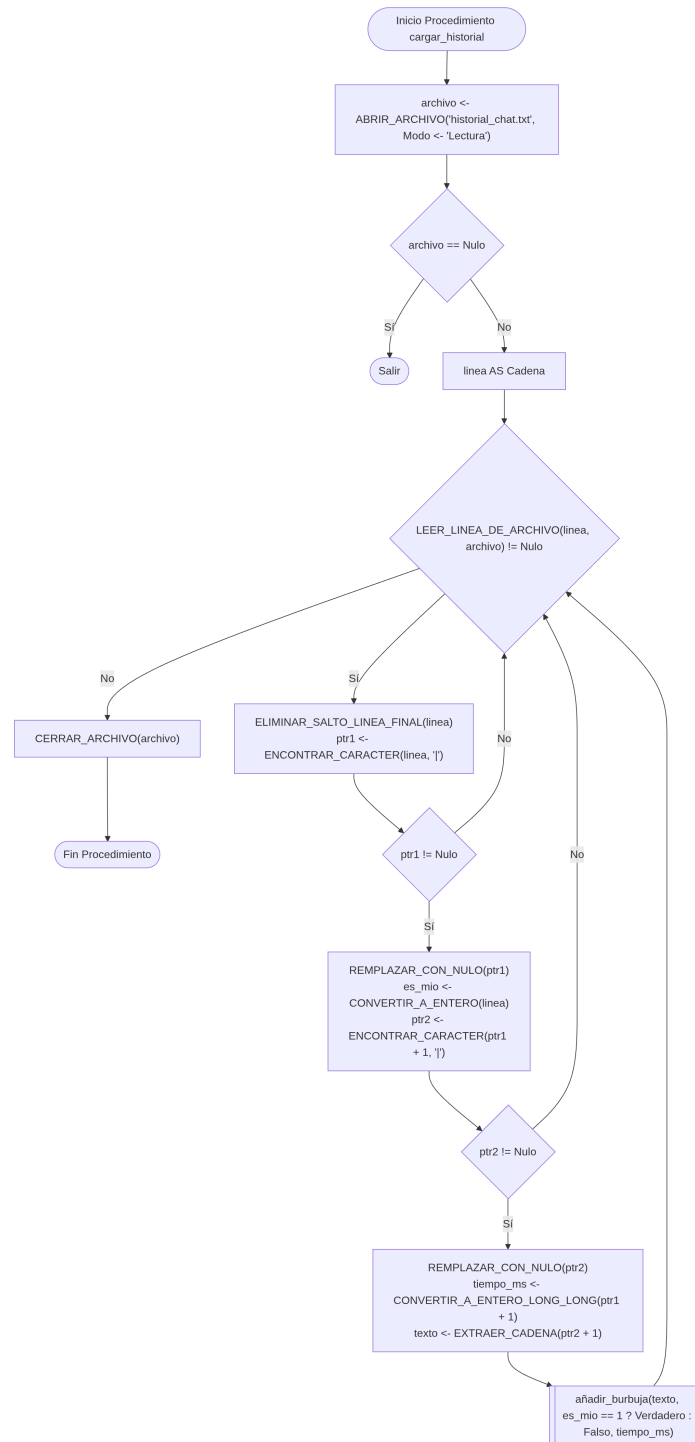
## 6. Procedimiento `ventana_destruida()`



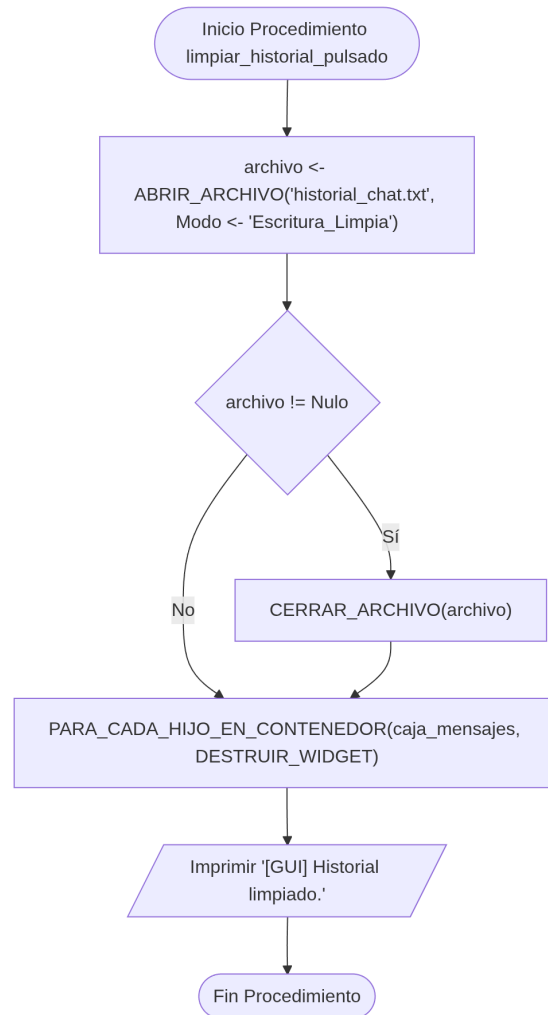
## 7. Procedimiento guardar\_en\_historial()



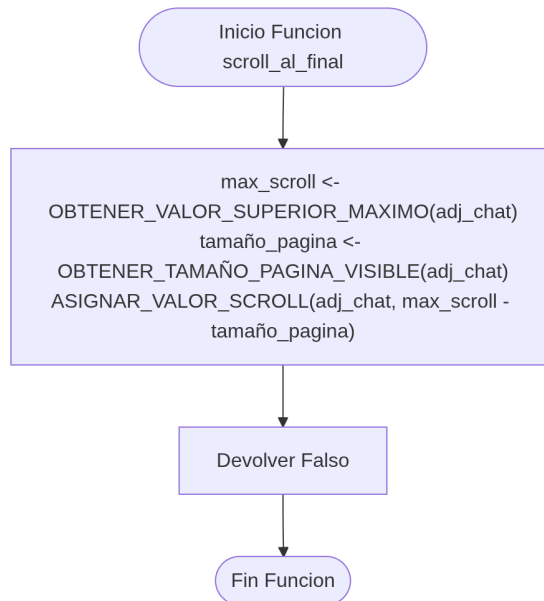
## 8. Procedimiento cargar\_historial()



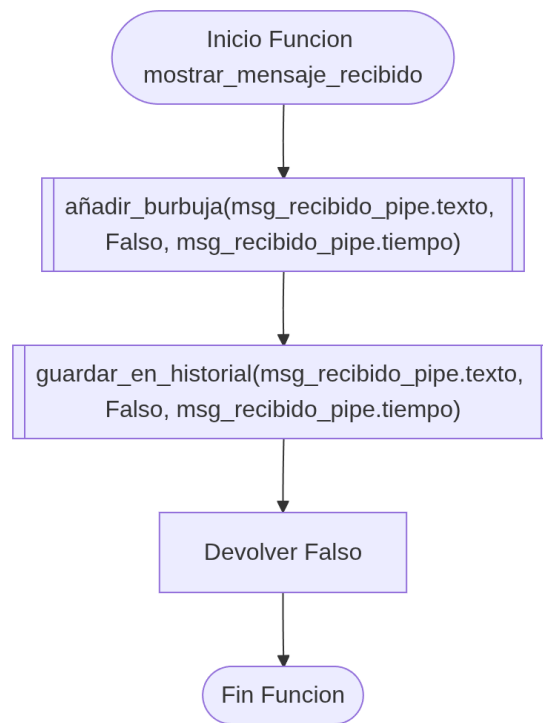
## 9. Procedimiento limpiar\_historial\_pulsado()



## 10. Función `scroll_al_final()`

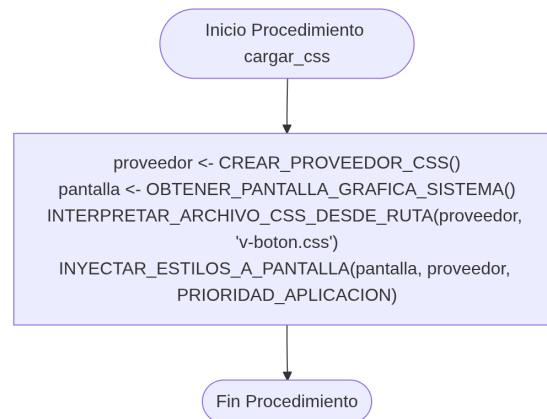


## 11. Función `mostrar_mensaje_recibido()`





## 12. Procedimiento cargar\_css()



### 4.4 Archivo Principal (chat.c)

Este archivo es el orquestador de toda la lógica de la conexión P2P y la interfaz gráfica que se muestra al usuario. Al modular ambas partes, se requiere la bifurcación del sistema en dos procesos independientes (un padre y un hijo) mediante la directiva `fork()`, lo que permite manejar la aplicación completa de forma concurrente sin que la red bloquee la interfaz.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5 #include "header.h"
6
7 // Tubería que se usa para comunicar los dos procesos (Pipe A y Pipe B)
8 Tuberia principal;
9
10 int main(int argc, char *argv[])
11 {
12     // Creamos las dos tuberías a utilizar durante la ejecución
13     if (pipe(principal.B) < 0 || pipe(principal.A) < 0)
14     {
15         perror("Error al crear los pipes");
16         return 1;
17     }
18
19     // Creamos un proceso hijo mediante fork para levantar la conexión P2P
20     pid_t process = fork();
21
22     // Si ocurre algún error en la bifurcación, se cancela la ejecución
```

```
23  if (process < 0)
24      return 1;
25
26  // De lo contrario, el proceso hijo (process == 0) se encarga de la conexión P2P
27  if (process == 0)
28  {
29      // El hijo cierra los extremos no usados de las tuberías
30      close(principal.A[0]);
31      close(principal.B[1]);
32
33      // Llama a la función que levanta la red y los sockets
34      conexion(&principal);
35
36      // Cierra los extremos utilizados para terminar exitosamente y liberar memoria
37      close(principal.A[1]);
38      close(principal.B[0]);
39
40      exit(0);
41  }
42  else // El proceso padre se encarga de renderizar la interfaz gráfica
43  {
44      // Se duerme 2 segundos para dar tiempo a que el back-end levante el servidor
45      sleep(2);
46
47      // El padre cierra los extremos que no utiliza de las tuberías
48      close(principal.A[1]);
49      close(principal.B[0]);
50
51      // Llama a la función principal de GTK que crea la vista
52      interfaz_grafica(argc, argv, &principal);
53
54      // Cierra los extremos utilizados tras cerrar la ventana
55      close(principal.A[0]);
56      close(principal.B[1]);
57
58      // El padre espera a que el proceso hijo termine para cerrarse exitosamente
59      wait(NULL);
60  }
61  return 0;
62 }
```

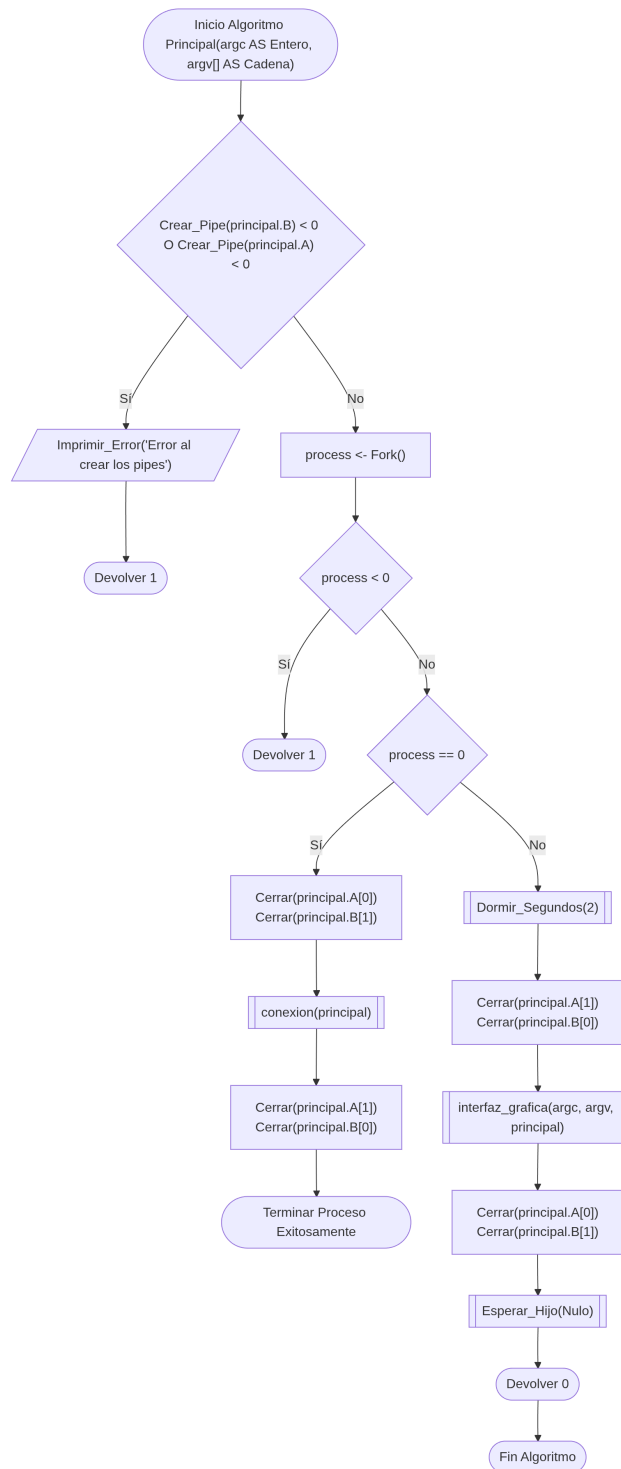
#### 4.4.1 Pseudocódigo (chat.c)

```
1 // Instanciación global de los canales de comunicación cruzados
2 Tuberia principal
3
4 Algoritmo Principal(argc AS Entero, argv[] AS Cadena):
5
6     // 1. Crear las dos tuberías IPC (Inter-Process Communication)
7     // Pipe A: Backend -> Interfaz | Pipe B: Interfaz -> Backend
8     Si Crear_Pipe(principal.B) < 0 O Crear_Pipe(principal.A) < 0 Entonces
9         Imprimir_Error("Error al crear los pipes")
10        Devolver 1
11    Fin Si
12
13    // 2. Dividir la ejecución del programa en dos procesos paralelos
14    process <- Fork()
15
16    Si process < 0 Entonces
17        Devolver 1 // Cancelar todo si el fork falla
18    Fin Si
19
20    // ===== RAMIFICACIÓN DE PROCESOS =====
21
22    Si process == 0 Entonces
23        // -----
24        // CÓDIGO DEL PROCESO HIJO (Módulo Backend / Red P2P)
25        // -----
26
27        // Cierra los extremos invertidos que le corresponden al Padre
28        Cerrar(principal.A[0]) // No lee de A (él escribe en A[1])
29        Cerrar(principal.B[1]) // No escribe en B (él lee de B[0])
30
31        // Inicializa el entorno de red P2P y sus 4 hilos internos
32        conexion(principal)
33
34        // Limpieza de descriptores al apagar el backend
35        Cerrar(principal.A[1])
36        Cerrar(principal.B[0])
37
38        Terminar Proceso Exitosamente
39
40    Sino
41        // -----
42        // CÓDIGO DEL PROCESO PADRE (Módulo Frontend / Interfaz Gráfica GTK)
43        // -----
```

```
44
45 // Pausa de cortesía para permitir que el Backend abra los Sockets de red
46 Dormir_Segundos(2)
47
48 // Cierra los extremos invertidos que le corresponden al Hijo
49 Cerrar(principal.A[1]) // No escribe en A (él lee de A[0])
50 Cerrar(principal.B[0]) // No lee de B (él escribe en B[1])
51
52 // Lanza y bloquea la ejecución en la interfaz gráfica estilo MSN
53 interfaz_grafica(argc, argv, principal)
54
55 // Limpieza de descriptores al cerrar la ventana de la GUI
56 Cerrar(principal.A[0])
57 Cerrar(principal.B[1])
58
59 // Sincronización final: Esperar a que el Hijo muera de forma segura
60 Esperar_Hijo(Nulo)
61
62 Fin Si
63
64 Devolver 0
65 Fin Algoritmo
```

#### 4.4.2 Diagrama de flujo (chat.c)

## Algoritmo principal main()



## 4.5 Conexión Física y Enrutamiento de Red (Tailscale)

Para establecer la comunicación peer to peer entre dos computadoras ubicadas en distintas redes geográficas, nos enfrentamos al problema clásico de las restricciones de red impuestas por los ruteadores y el NAT (*Network Address Translation*) de los proveedores de servicios de internet (ISP), los cuales bloquean las conexiones entrantes por defecto.

Para resolver este desafío de conectividad sin necesidad de realizar configuraciones invasivas como el *Port Forwarding* en cada módem, se implementó **Tailscale**. Esta herramienta es una VPN de malla (*Mesh VPN*) basada en el protocolo WireGuard que crea una red privada virtual superpuesta.

Al instalar Tailscale en ambas máquinas, el software les asigna una dirección IP estática dentro de una misma subred virtual (por ejemplo, en el rango 100.x.x.x). Esto permite que los *sockets* de nuestra aplicación en C puedan realizar el `bind()` y el `connect()` apuntando directamente a las direcciones IP virtuales proporcionadas por la plataforma, engañando al sistema para que interactúe como si ambas computadoras estuvieran conectadas al mismo conmutador (*switch*) en una red de área local (LAN). De este modo, garantizamos un túnel encriptado, directo y de baja latencia para la transmisión de las estructuras Mensaje.

## 5. Conclusiones

- Enrique Yoab Cortez Rios

El desarrollo de esta práctica representó un gran reto técnico, ya que la aplicación pasó por múltiples etapas de rectificación y optimización de recursos. La implementación de tuberías cruzadas (pipes) y la asignación de hilos específicos para la lectura y escritura entre el front-end y el back-end resultó ser la solución más eficiente. Esta arquitectura nos permitió comunicar los procesos de forma directa y asíncrona, evitando el uso de intermediarios para transferir los datos entre la red y la interfaz.

Por otro lado, la biblioteca GTK fue fundamental para construir una interfaz gráfica nativa íntegramente en C. Una gran ventaja de esta herramienta fue la posibilidad de estilizar la ventana mediante un archivo CSS. Al tener experiencia previa con el desarrollo web y la estructura de componentes visuales, integrar estas hojas de estilo resultó un proceso muy natural, permitiendo dotar al chat de una apariencia mucho más atractiva, moderna y amigable para el usuario.

Asimismo, la implementación de Tailscale fue un gran acierto para la arquitectura del proyecto, ya que nos permitió establecer una conexión peer-to-peer a través de internet, enlazando ambas computadoras mediante direcciones IP virtuales sin la estricta necesidad de estar conectadas físicamente por un cable Ethernet en la misma red local.

En conclusión, la práctica se completó con éxito y cumplió sus objetivos formativos. Se consolidaron los conceptos de programación concurrente, logrando distribuir la carga de trabajo entre los procesos para mantener todo el sistema coordinado. El uso de hilos (pthreads) demostró ser clave para manejar las operaciones bloqueantes de escucha y escritura en las tuberías y los sockets, garantizando que la interfaz gráfica se mantuviera reactiva en todo momento y libre de congelamientos.

- **Mario Mendoza González**

El desarrollo de este proyecto permitió entender de una forma mucho más práctica cómo funcionan varios de los conceptos vistos en la materia cuando se integran en una sola aplicación. Aunque al inicio temas como la comunicación entre procesos, el uso de hilos o la programación con sockets parecían independientes, durante la implementación fue posible ver cómo todos ellos se relacionan para resolver un mismo problema.

Uno de los mayores retos fue lograr que la interfaz gráfica, la comunicación mediante tuberías y la conexión P2P trabajaran al mismo tiempo sin interferirse entre sí. Esto ayudó a comprender la importancia de la concurrencia y de una buena organización del sistema, ya que cada proceso y cada hilo tienen una función específica dentro de la aplicación.

Además del aspecto técnico, el proyecto permitió acercarse al desarrollo de software de una manera más completa, considerando no solo que el programa funcionara, sino también que fuera estable, fácil de utilizar y capaz de mantener la información de las conversaciones. Entender cómo un mensaje viaja desde la interfaz, pasa por los mecanismos de comunicación interna y finalmente llega a otro nodo hizo más claro el funcionamiento de un sistema distribuido real (una aplicación tipo messenger).

Finalmente, quedó claro que detrás de las aplicaciones de mensajes existe una gran cantidad de procesos trabajando. La experiencia obtenida durante el desarrollo ayudó a reforzar los conocimientos de la materia y a comprender mejor lo que implica diseñar sistemas.

## 6. Referencias

- Oporto Díaz, S. (s.f.). Material Adicional (SOSD – Mod 4): Concurrencia. Exclusión mutua y sincronización [PDF]. Departamento de Ciencias e Ingeniería de la Computación, Universidad Nacional del Sur. Disponible en [cs.uns.edu.ar](http://cs.uns.edu.ar).
- De Luz, S. (2026, 12 marzo). Tailscale, conecta equipos a una red privada virtual (VPN) fácilmente. RedesZone. Disponible en [RedesZone](https://redeszone.com).
- Purita, G. (2026, 17 marzo). ¿Qué son las redes P2P y cuál su utilidad? OBS Business School. Disponible en [OBS Business School](https://obsbusinessschool.com).
- Hilo en computación: ¿Qué es? — Lenovo México. (s. f.). Disponible en [Lenovo México](https://lenovo.com/mx).
- Klie, F. R. (2026, 5 marzo). Understanding ‘fork()’ in Linux: How Process Creation Really Works. DEV Community. Disponible en [DEV Community](https://dev.to).
- Team, G. (s. f.). The GTK Project - A free and open-source cross-platform widget toolkit. The GTK Team. Web oficial GTK